



PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA

MALTA

Machine Learning Theory
and Applications Lab

Aprendizado Profundo II

Revisão Transformers

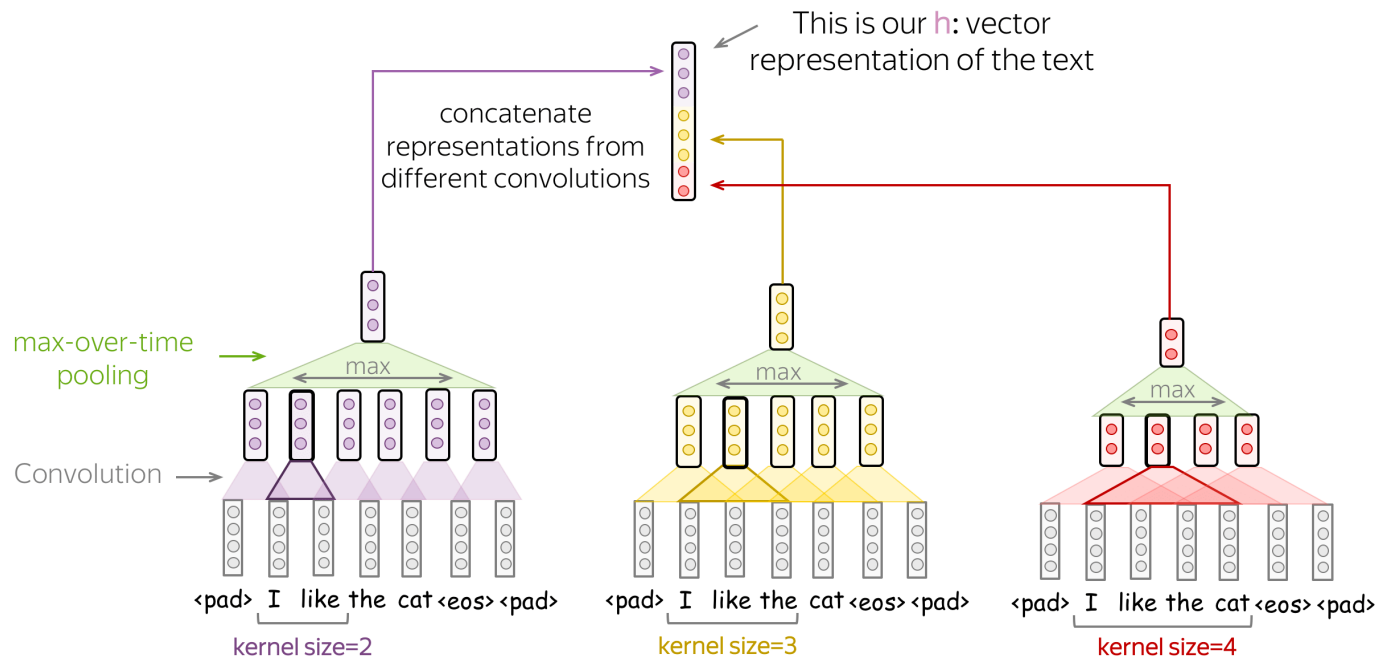
Prof. Me. Otávio Parraga

Roteiro

- Redes Recorrentes
 - RNNs
 - Sequence2Sequence
- Attention
 - Scaled Dot Product Attention
 - Scaled Dot Product Self-Attention
- Transformers

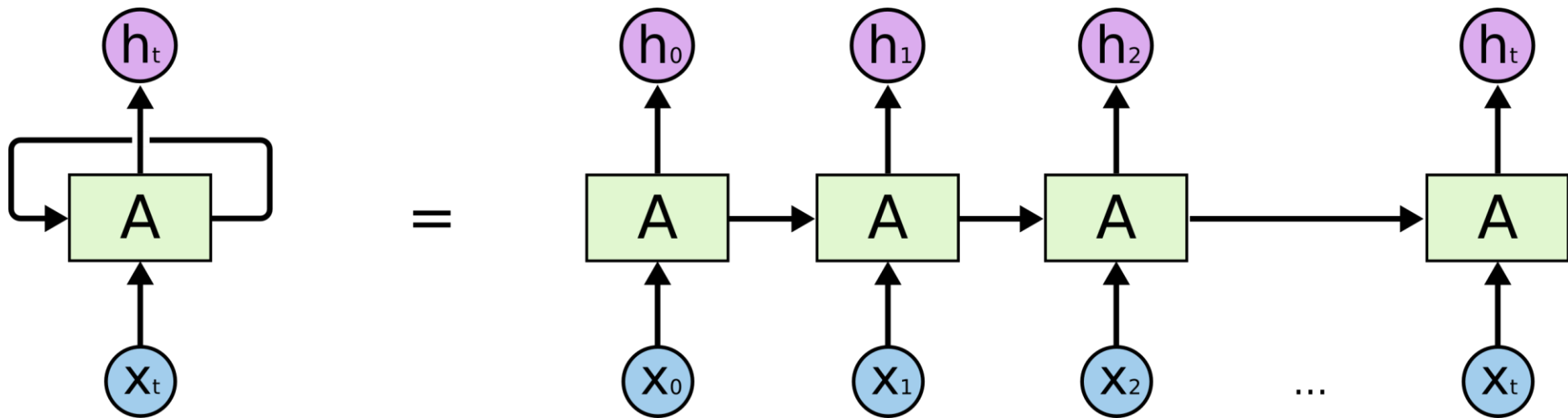
RNNs

- Em tarefas textuais, temos informação temporal
- Arquiteturas como MLP ou CNN não conseguem carregar temporalidade



RNNs

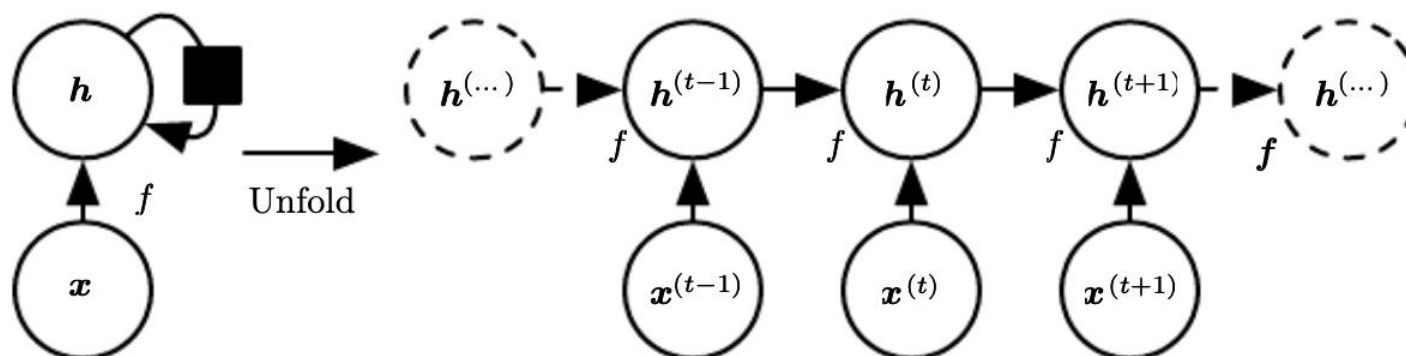
- Alternativa: RNNs



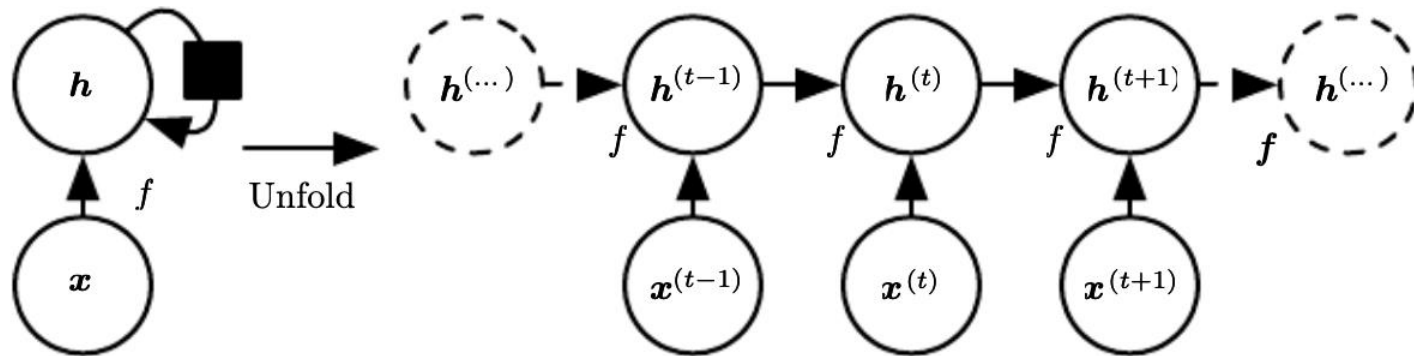
- Simulam a memória por meio do *Hidden State*

RNNs

- Na prática, um mesmo conjunto de matrizes é usada para **todas as operações**
- Pesos compartilhados entre diferentes *timesteps*



RNNs



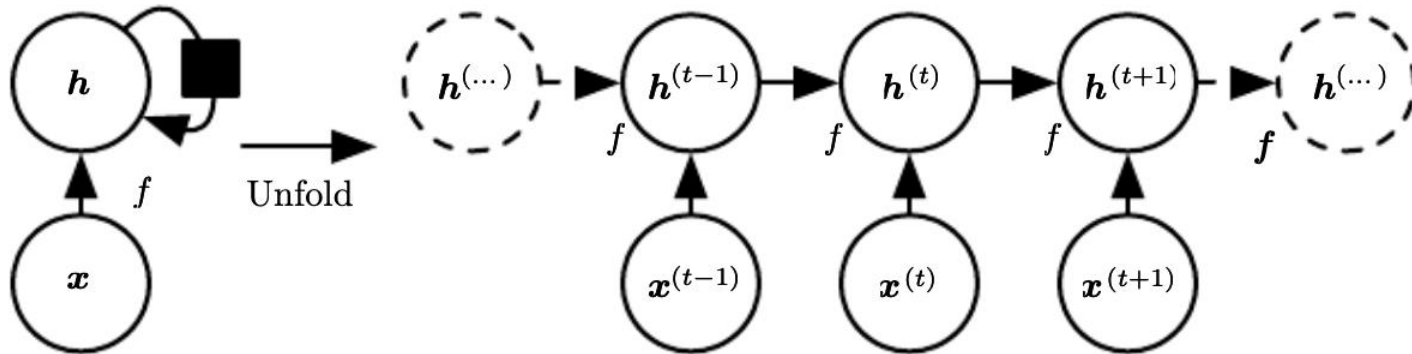
$$a^{(t)} = b + W h^{(t-1)} + U x^{(t)},$$

$$h^{(t)} = \tanh(a^{(t)}),$$

$$o^{(t)} = c + V h^{(t)},$$

$$\hat{y}^{(t)} = \text{softmax}(o^{(t)}),$$

RNNs



$$a^{(t)} = b + W h^{(t-1)} + U x^{(t)},$$

$$h^{(t)} = \tanh(a^{(t)}),$$

$$o^{(t)} = c + V h^{(t)},$$

$$\hat{y}^{(t)} = \text{softmax}(o^{(t)}),$$

Quando queremos ter uma saída em qualquer uma das camadas!

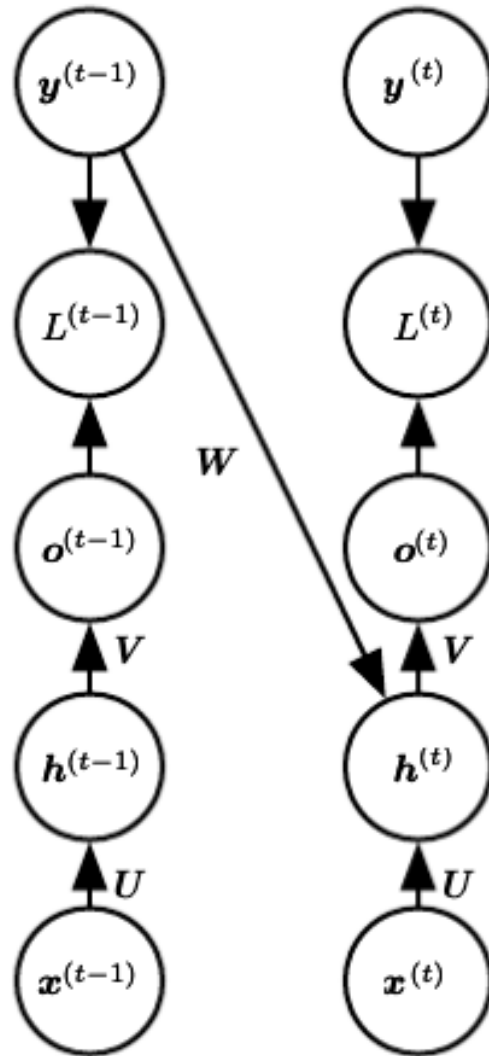
RNNs

$$a^{(t)} = b + W h^{(t-1)} + U x^{(t)},$$

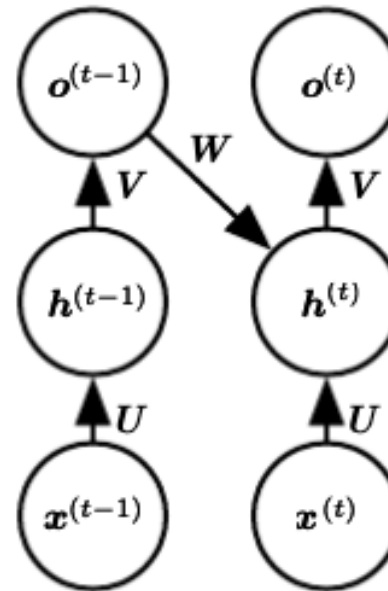
$$h^{(t)} = \tanh(a^{(t)}),$$

$$o^{(t)} = c + V h^{(t)},$$

$$\hat{y}^{(t)} = \text{softmax}(o^{(t)}),$$



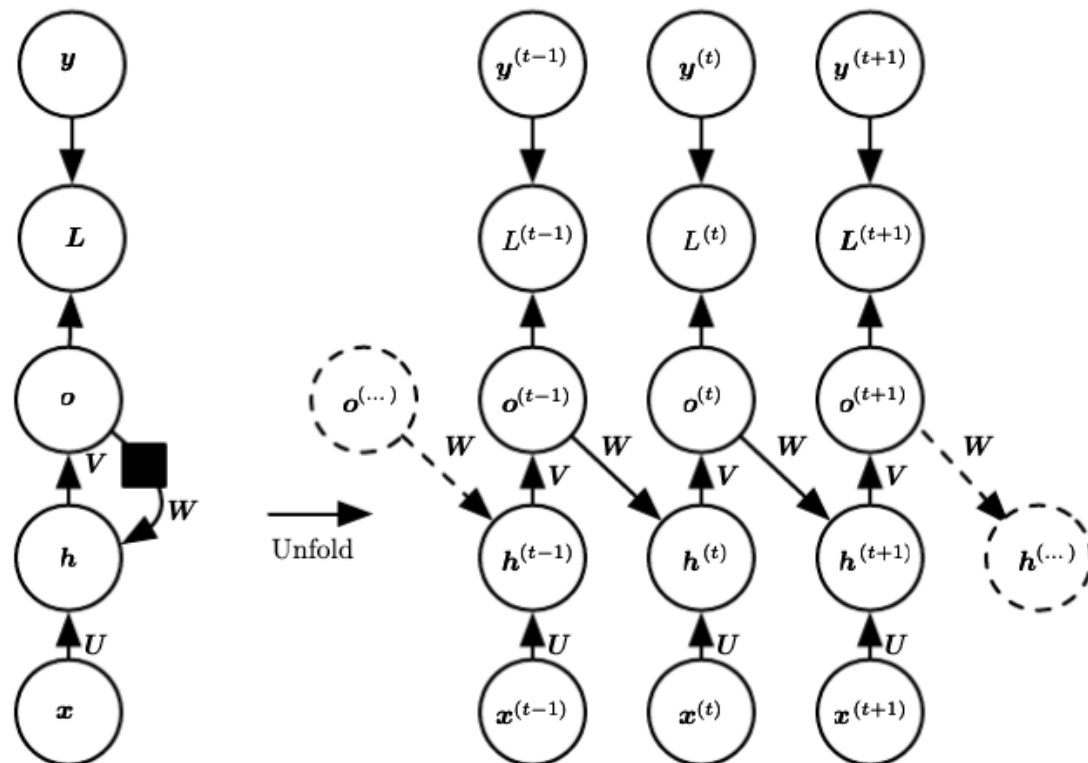
Train time



Test time

RNNs

- Para o processo de treinamento, utilizamos uma modificação do backpropagation, chamado ***Backpropagation Through Time (BPTT)***



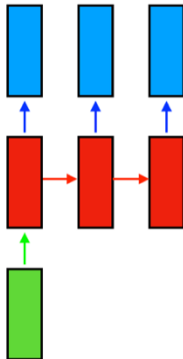
RNNs

- Na prática, podemos ajustar RNNs para **diferentes tarefas** mudando a arquitetura

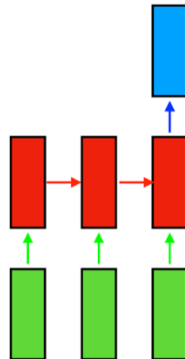
one to one



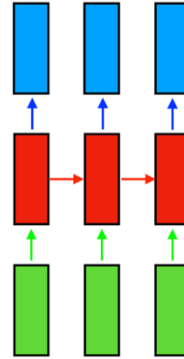
one to many



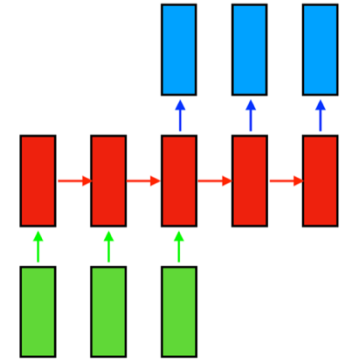
many to one



many to many



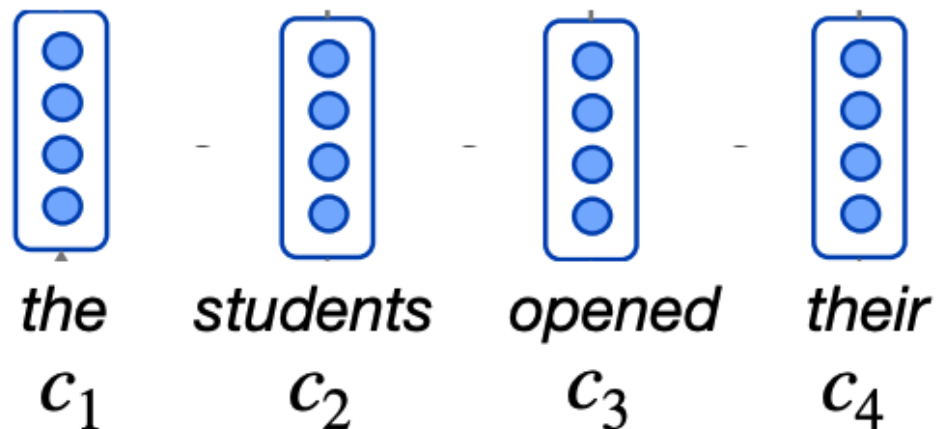
many to many



RNNs

- Para tarefas como modelagem de linguagem:

WordEmbeddings $\rightarrow c_1, c_2, c_3, c_4$



RNNs

WordEmbeddings $\rightarrow c_1, c_2, c_3, c_4$

HiddenStates $\rightarrow h^{(0)}$

Primeiro HS é aleatório!



the
 c_1



students
 c_2



opened
 c_3

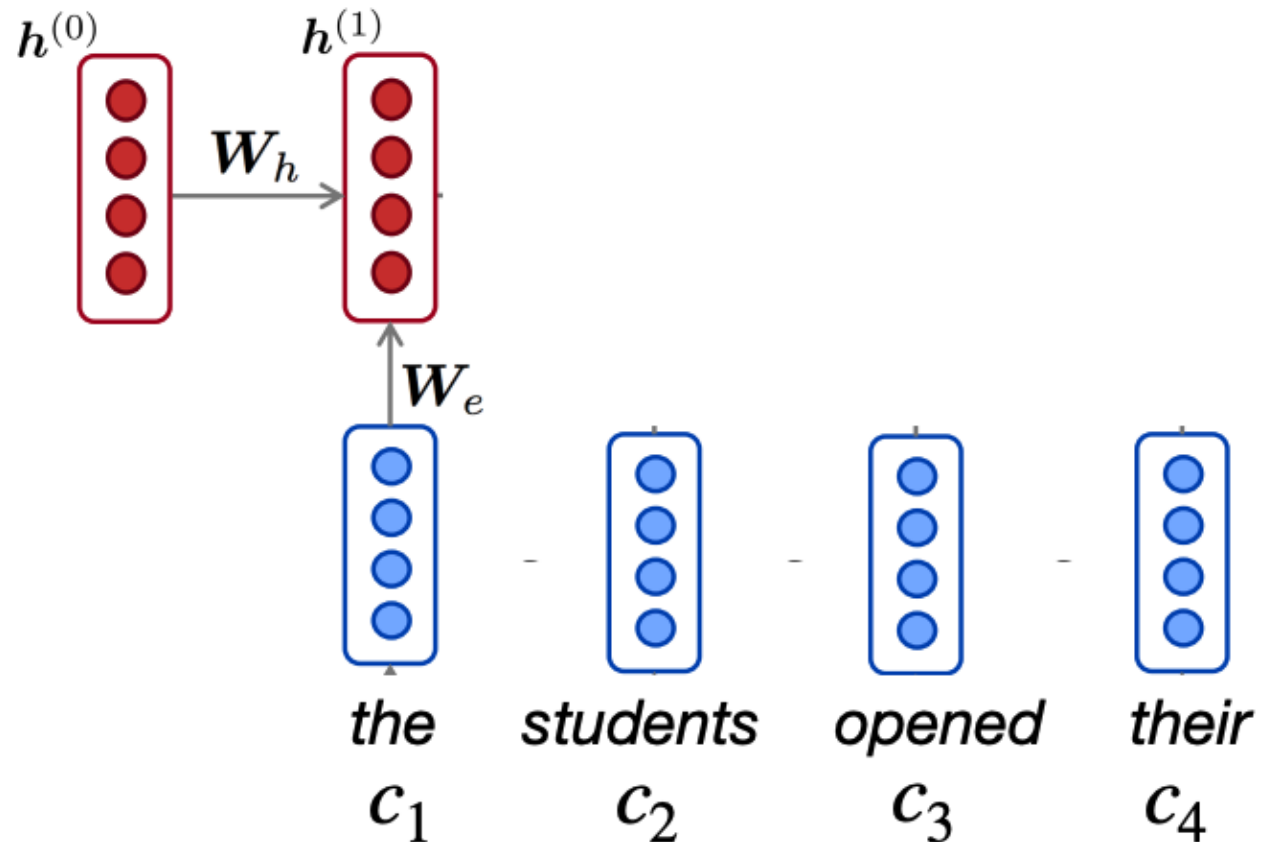


their
 c_4

RNNs

WordEmbeddings $\rightarrow c_1, c_2, c_3, c_4$

HiddenStates $\rightarrow h^{(0)}, h^{(1)}$

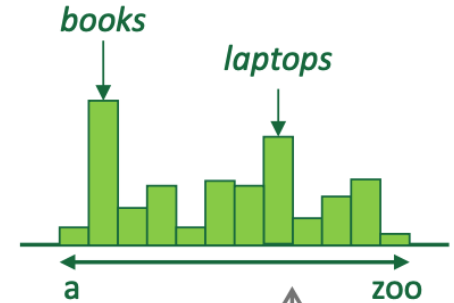


RNNs

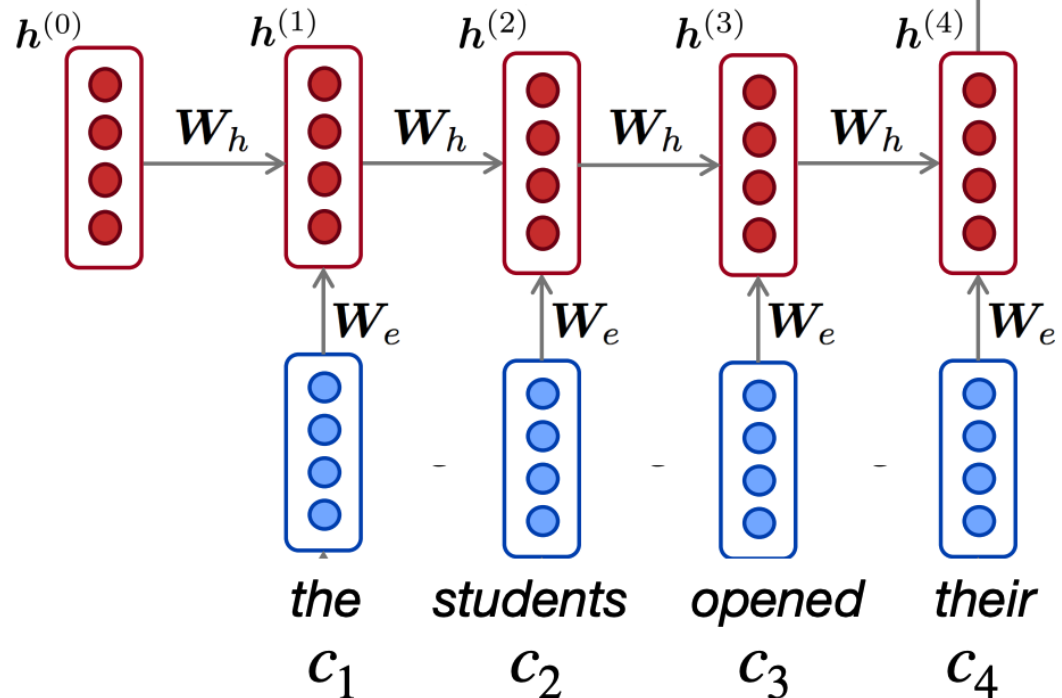
WordEmbeddings $\rightarrow c_1, c_2, c_3, c_4$

HiddenStates $\rightarrow h^{(0)}, h^{(1)}, h^{(2)}, h^{(3)}, h^{(4)}$

$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$



Onde W_2 é uma camada linear para mapear o HiddenState para uma palavra do vocabulário



RNNs

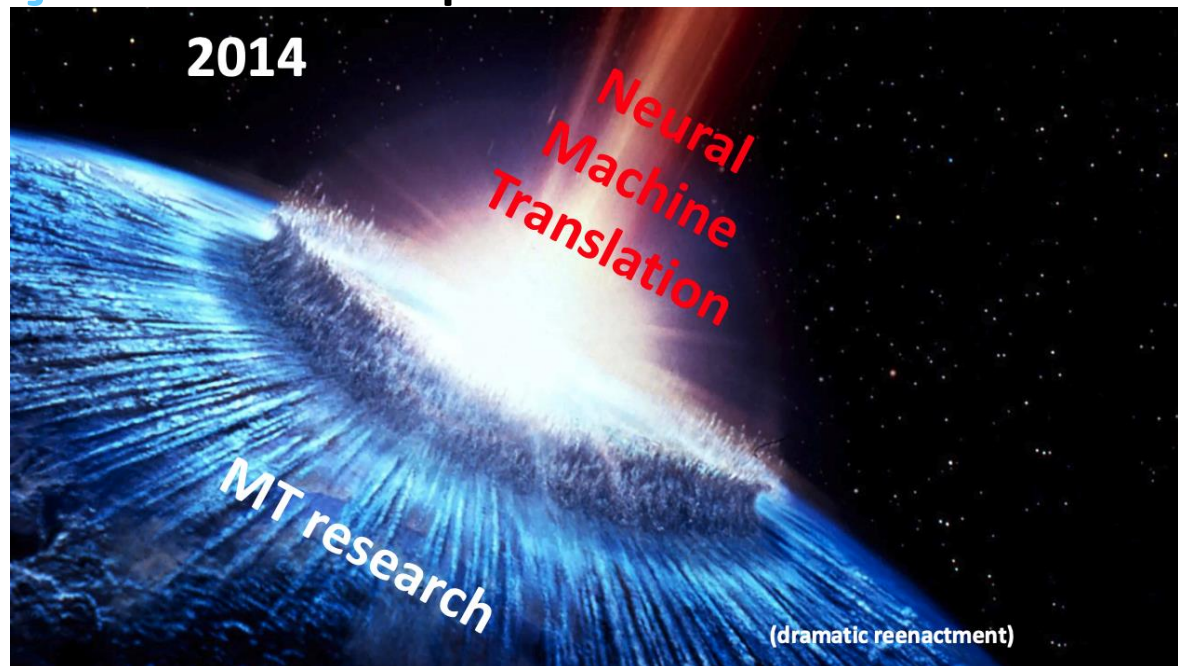
- Alguns problemas são clássicos:
 - Não podemos paralelizar (**estrutura sequencial**)
 - Problemas com **sequências grandes** (mesmo com as variações LSTM e GRU)
 - Problema do **Vanishing Gradient** por conta do BPTT
- Todavia, tivemos grandes avanços!

RNNs

- Vamos nos voltar para Tradução de Máquina
 - 1990s-2010s: **Statistical Machine Translation(SMT)**
 - Área grande de pesquisa
 - Sistemas complexos com inúmeras tabelas de frases equivalentes entre pares de linguagens
 - Esforço humano e linguístico
 - 2014: Surge estrutura **Sequence-to-Sequence (Seq2Seq)**
 - **Duas redes recorrentes**
 - Uma para “Compreender” e outra para “Gerar”

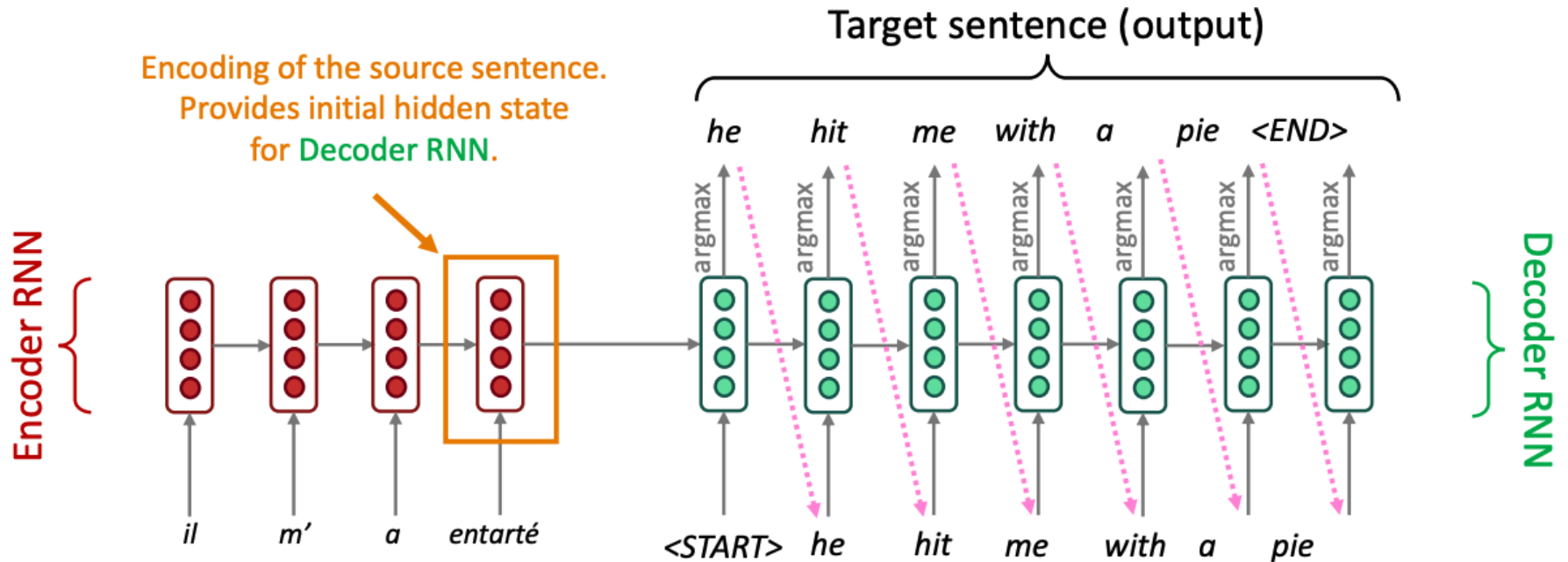
RNNs

- 2016: Redes Neurais se tornam o **método padrão** para tradução entre idiomas
- 2018: **Google muda a sua ferramenta de tradução** de SMT para NMT



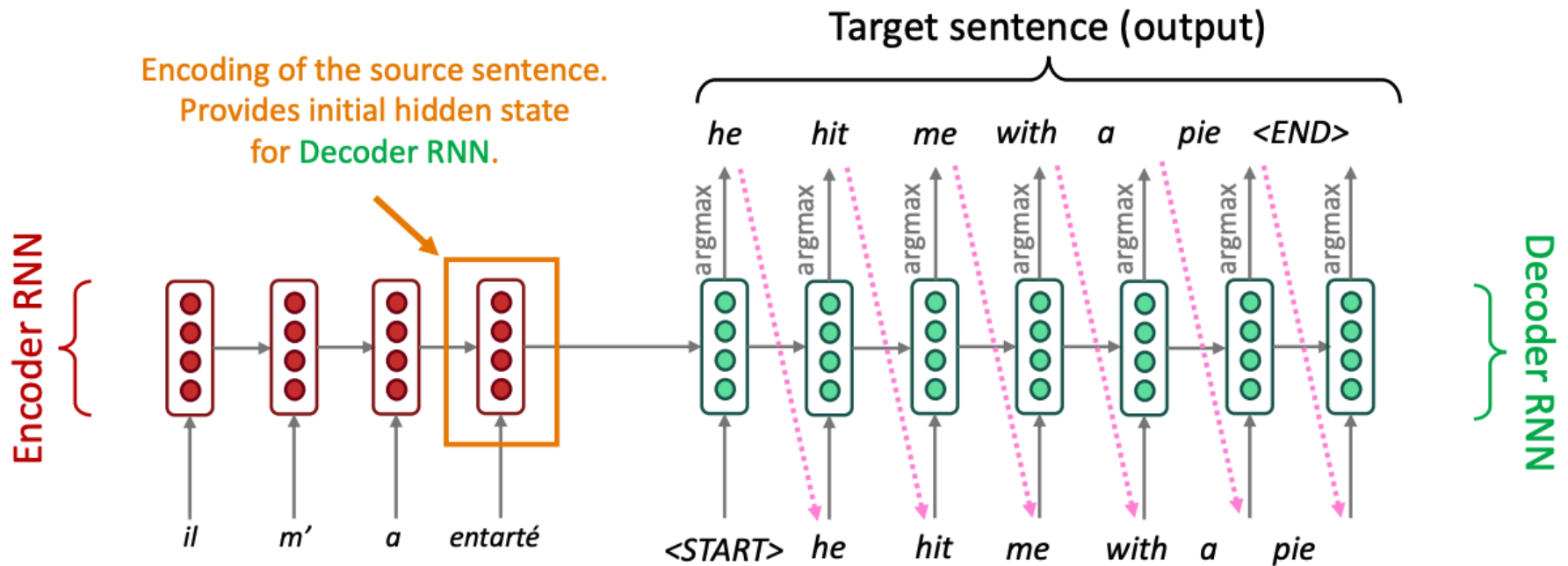
Sequence2Sequence

- Duas RNNs



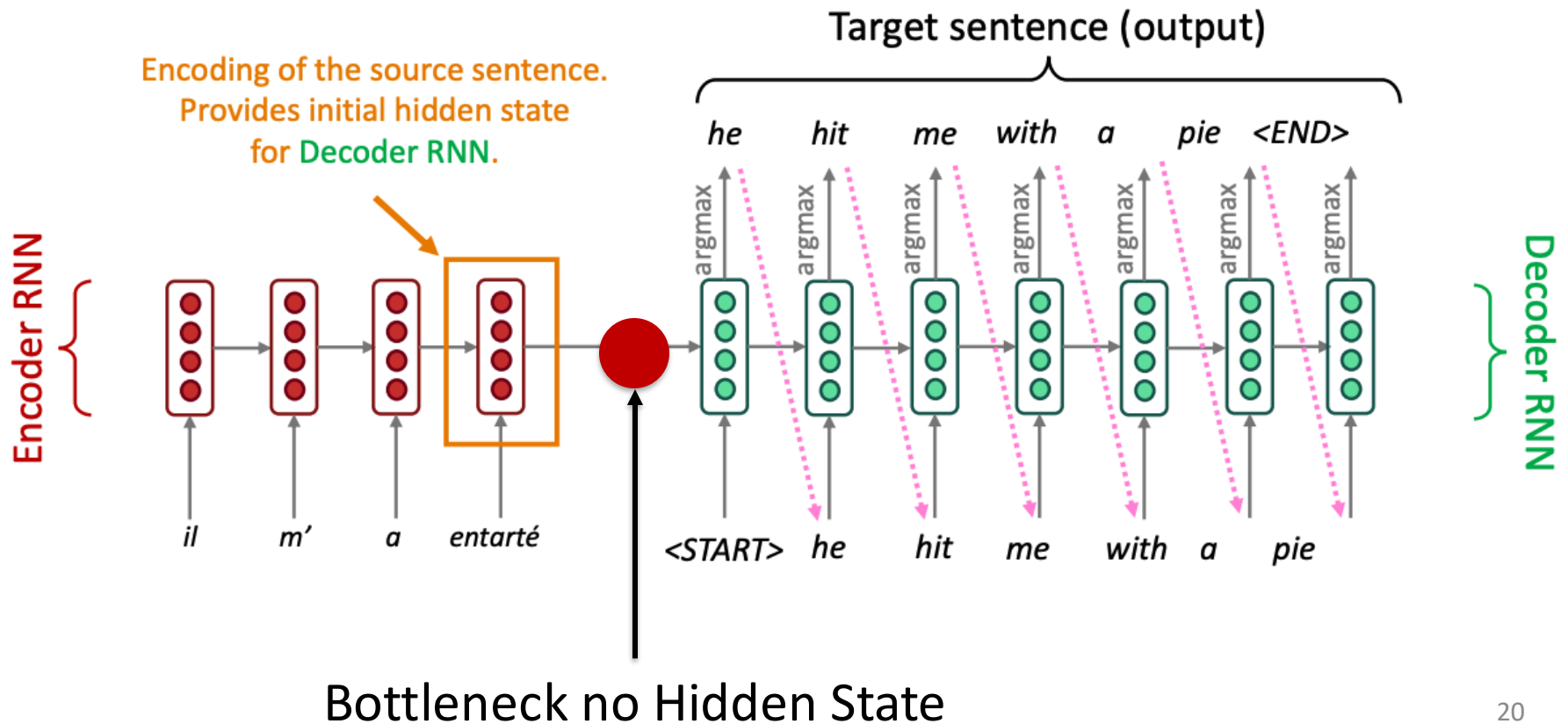
Sequence2Sequence

- O problema?



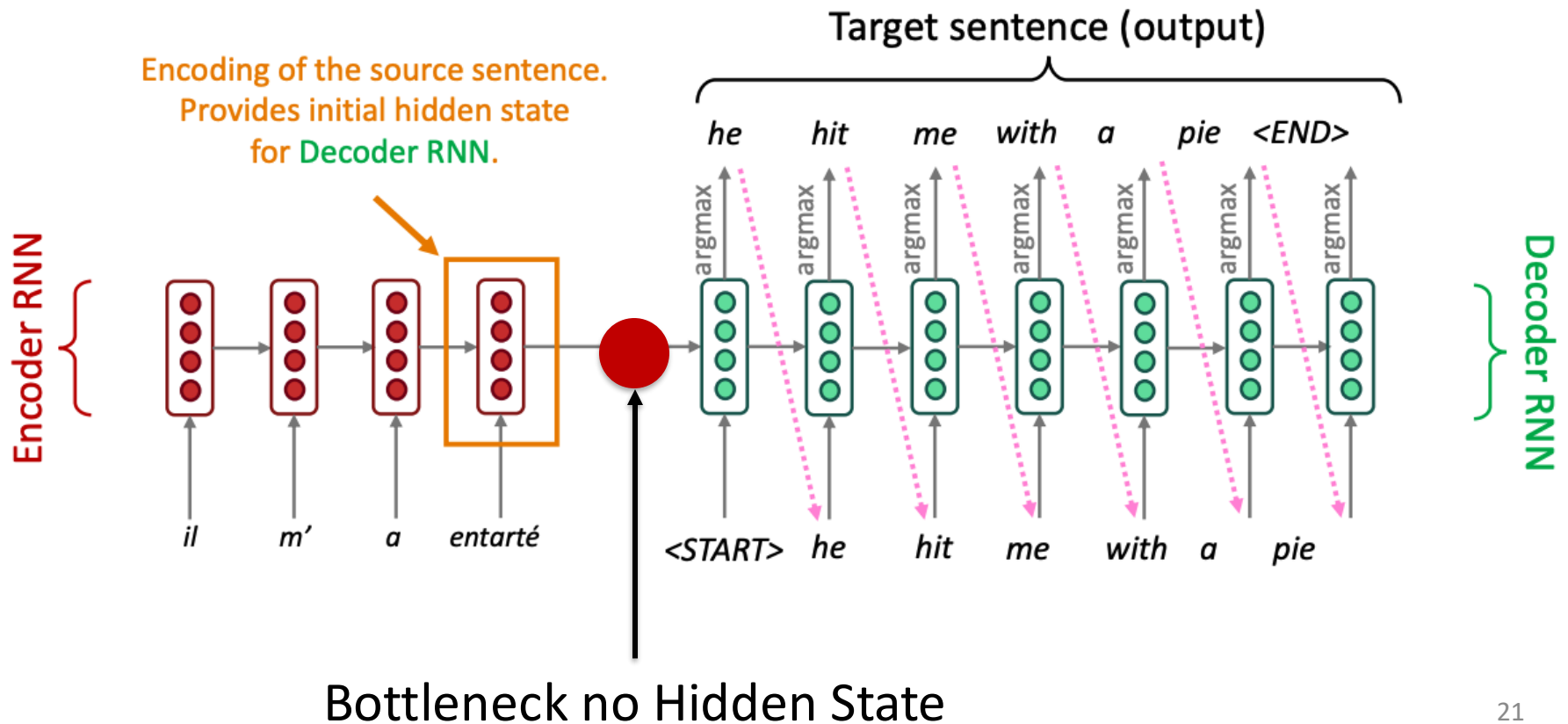
Sequence2Sequence

- O problema?



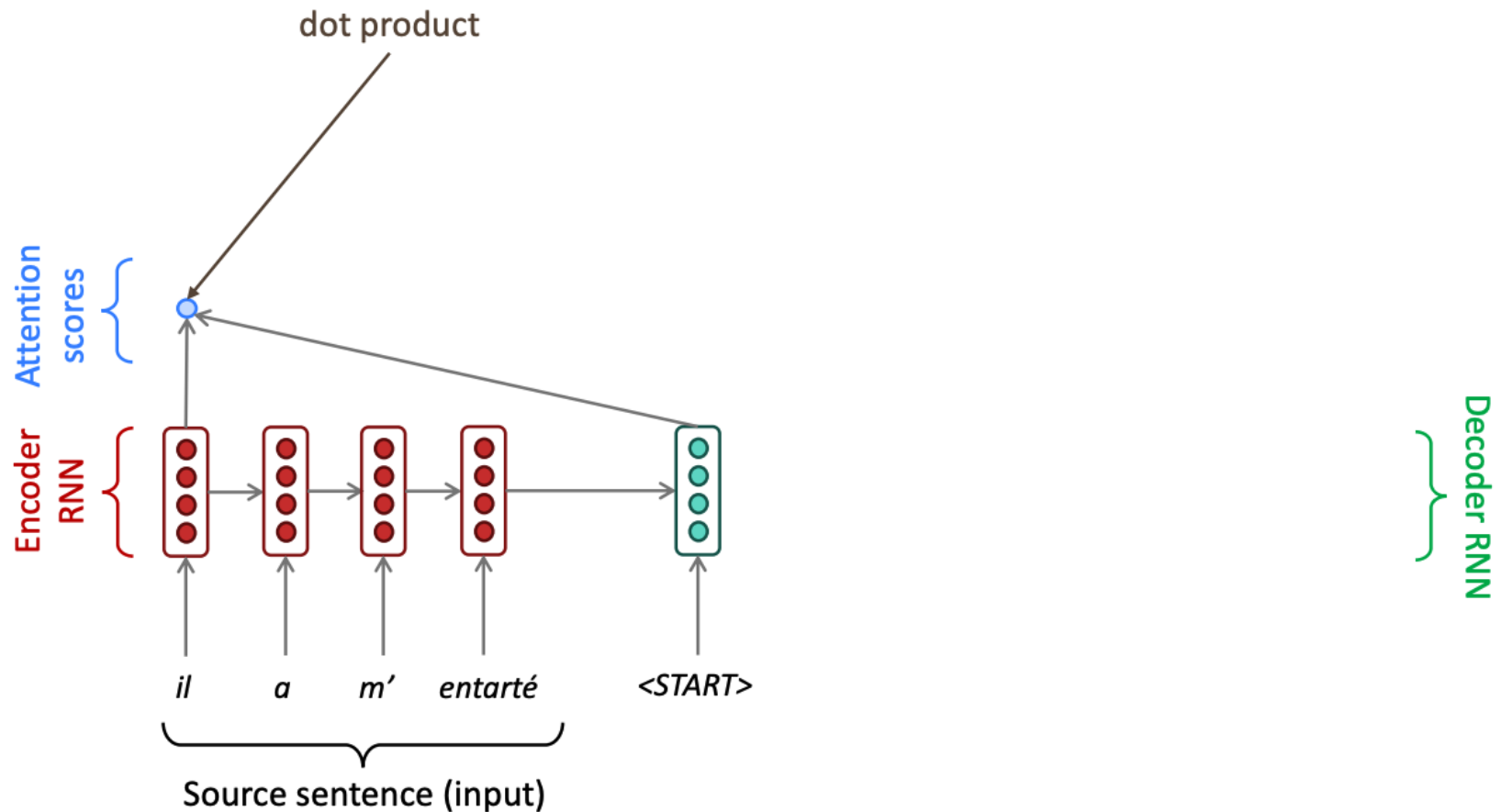
Sequence2Sequence

- Como resolver?



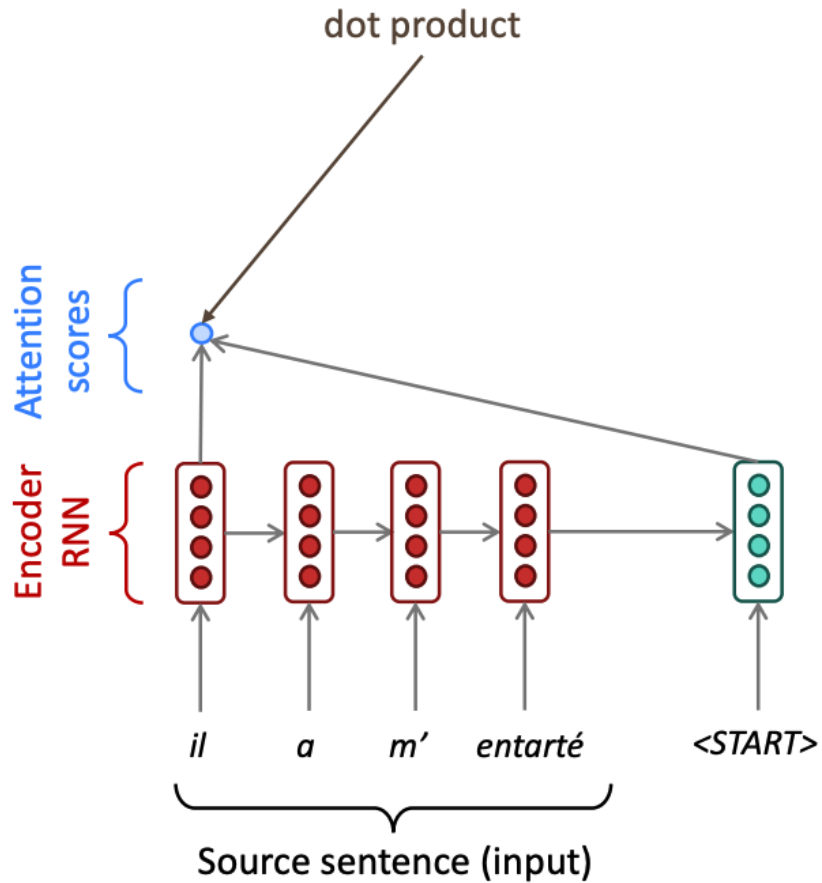
Sequence2Sequence

- Attention!

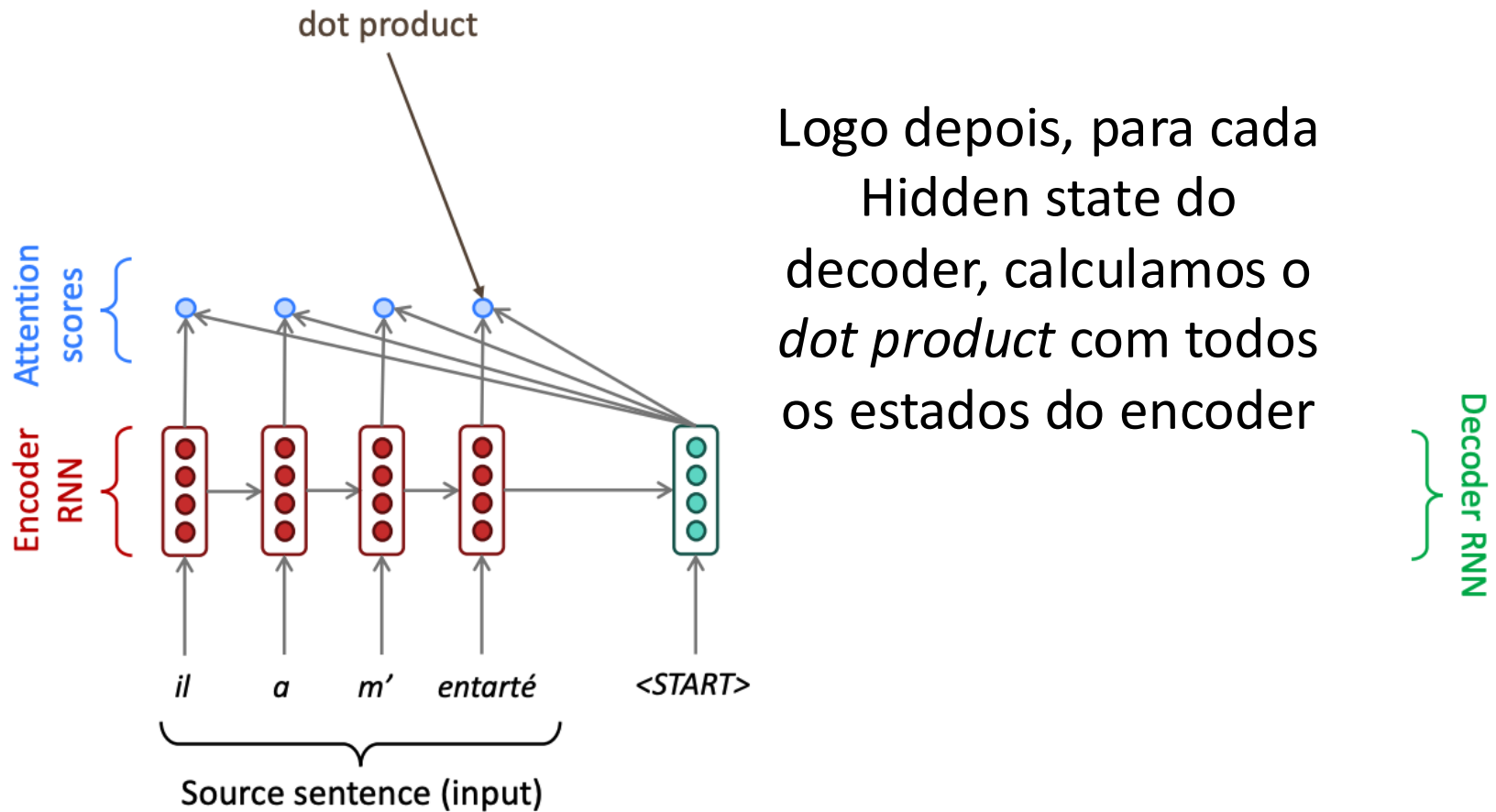


Attention

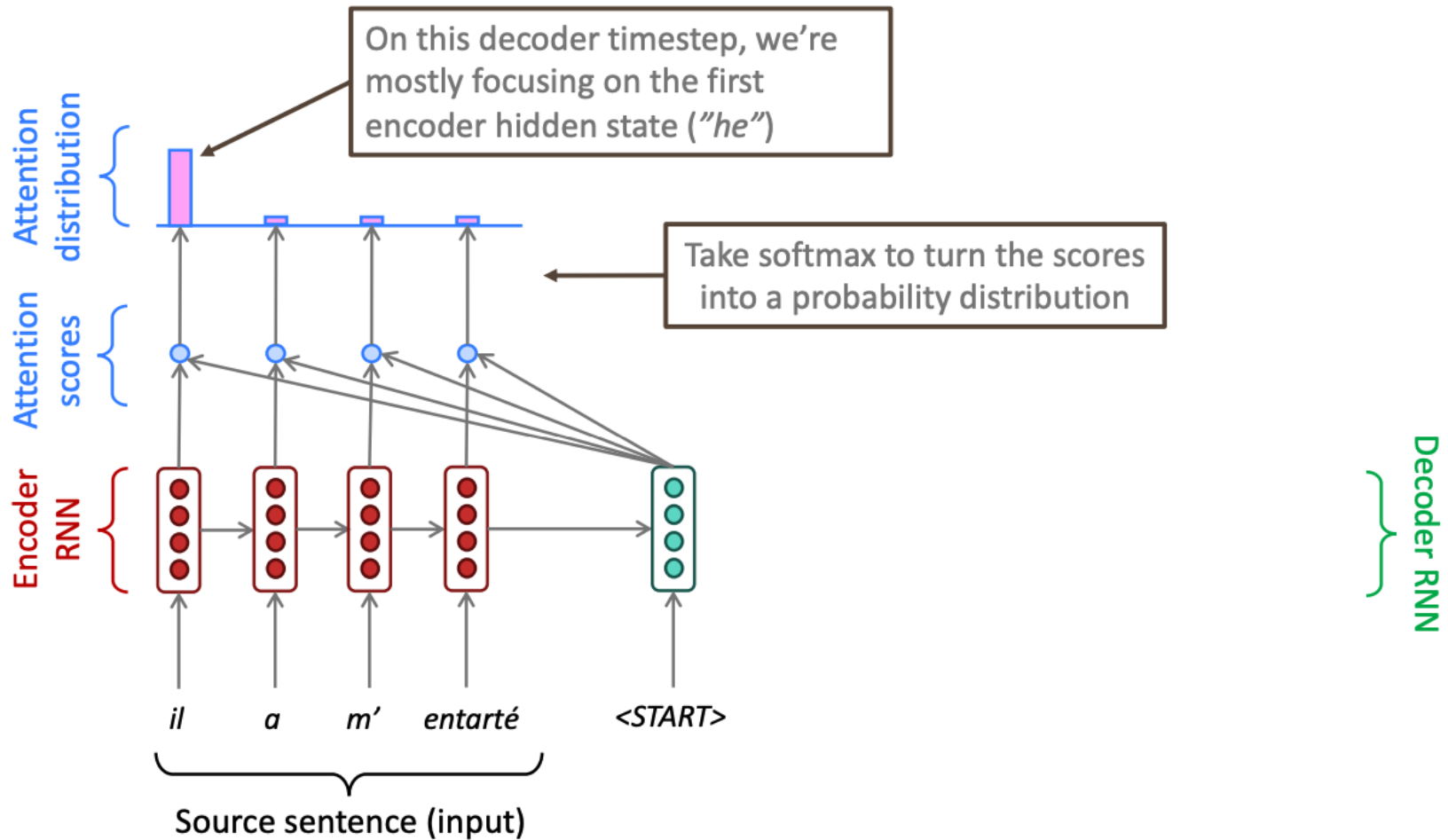
Salvamos todos os
Hidden states do
encoder!



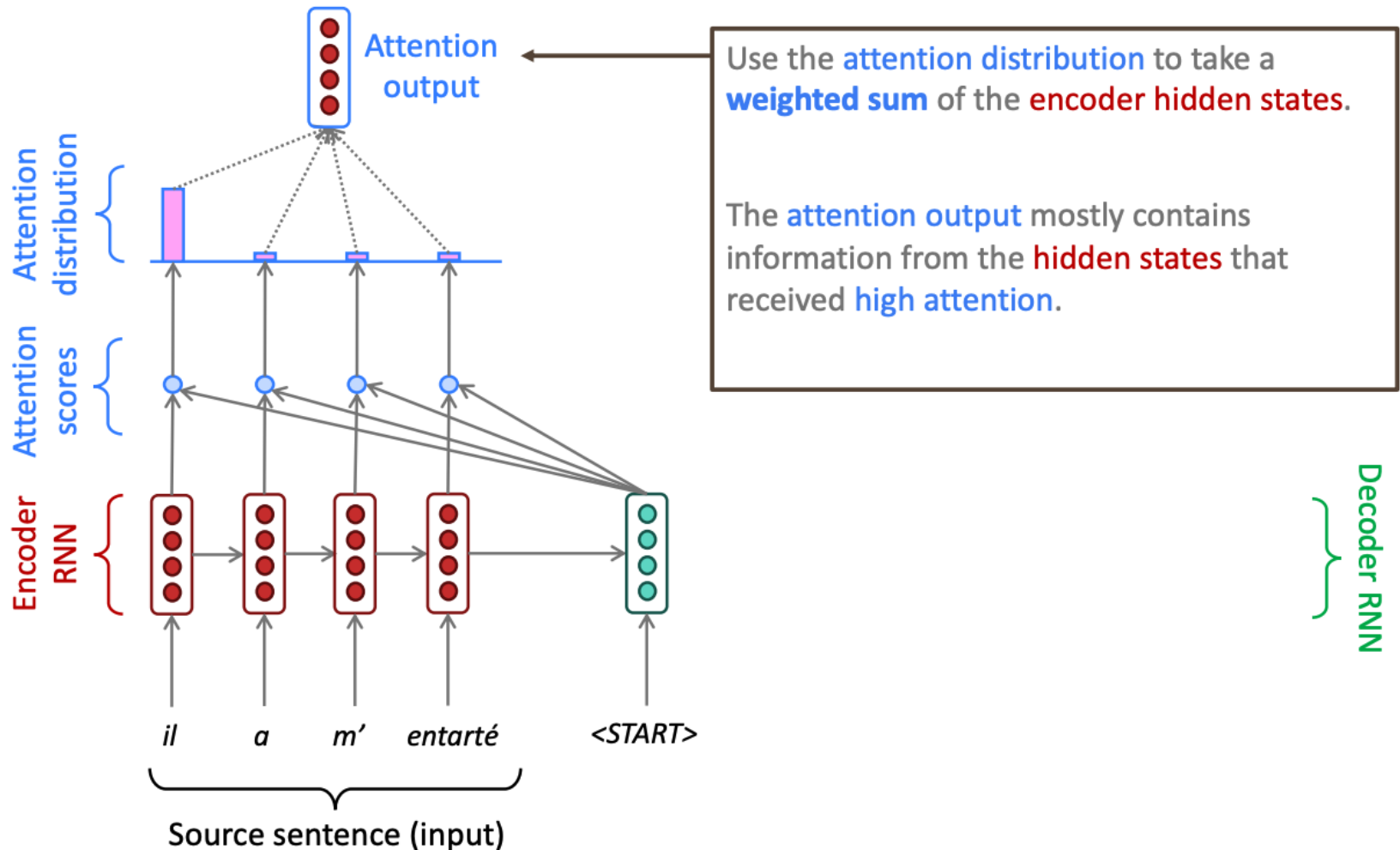
Attention



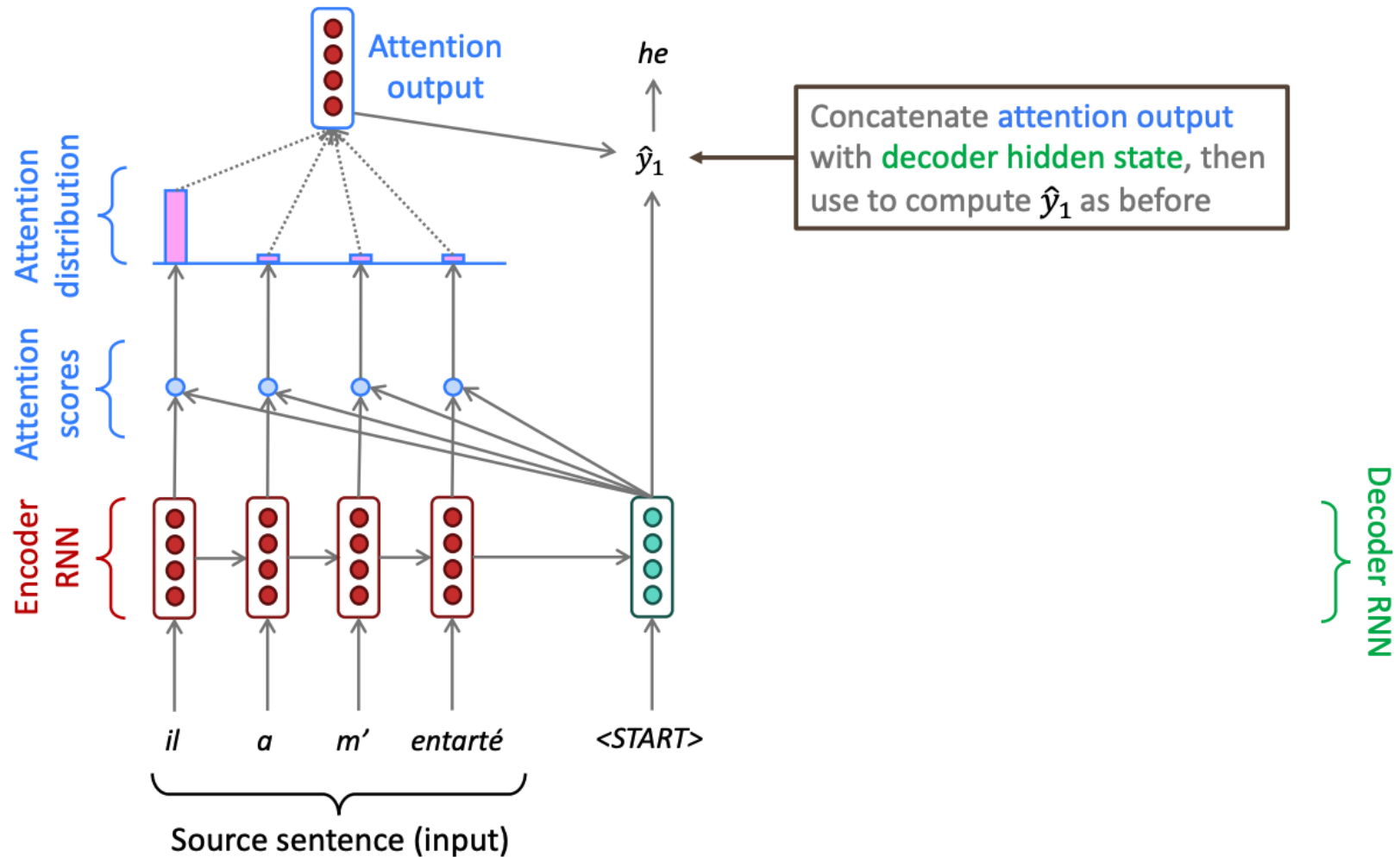
Attention



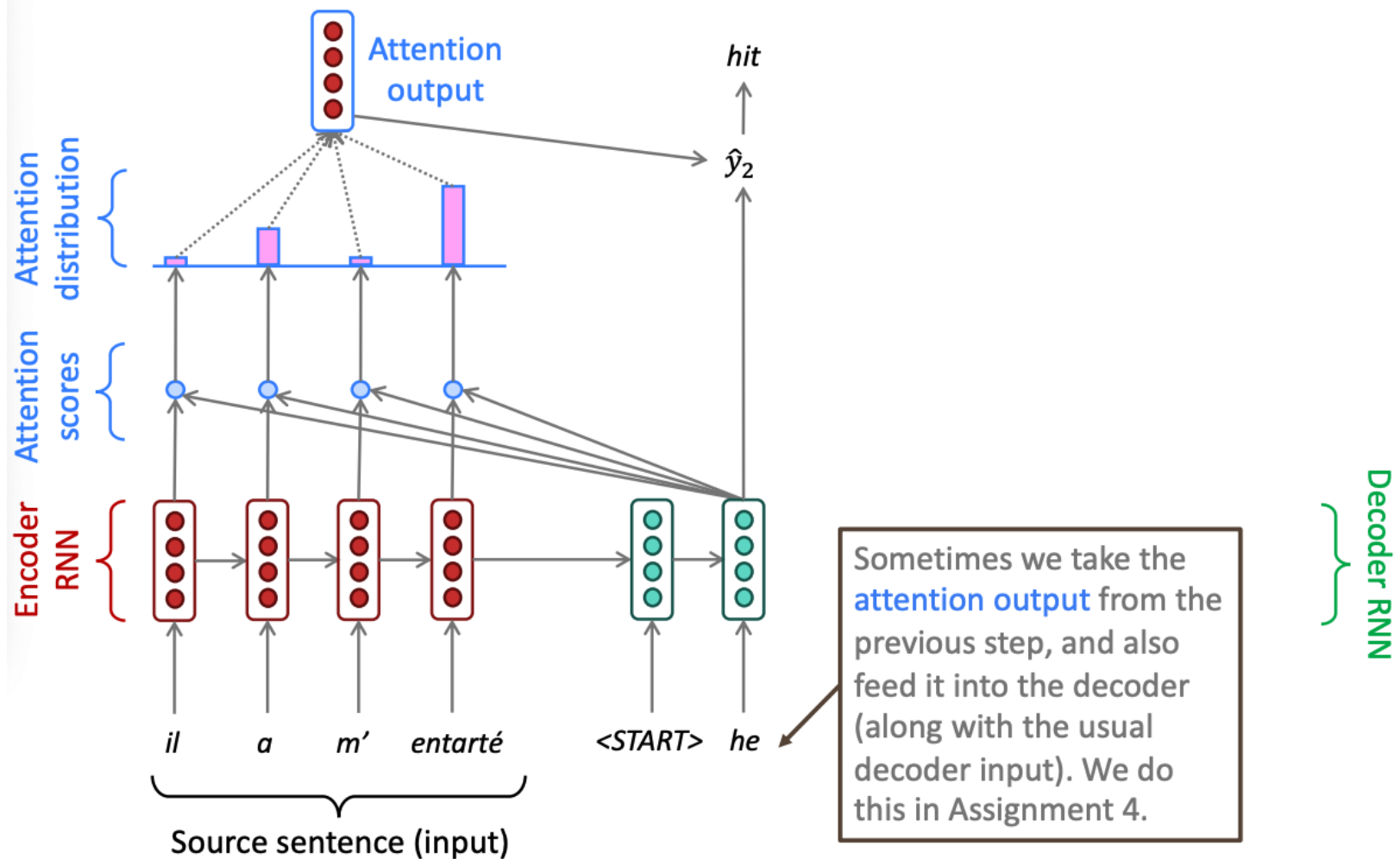
Attention



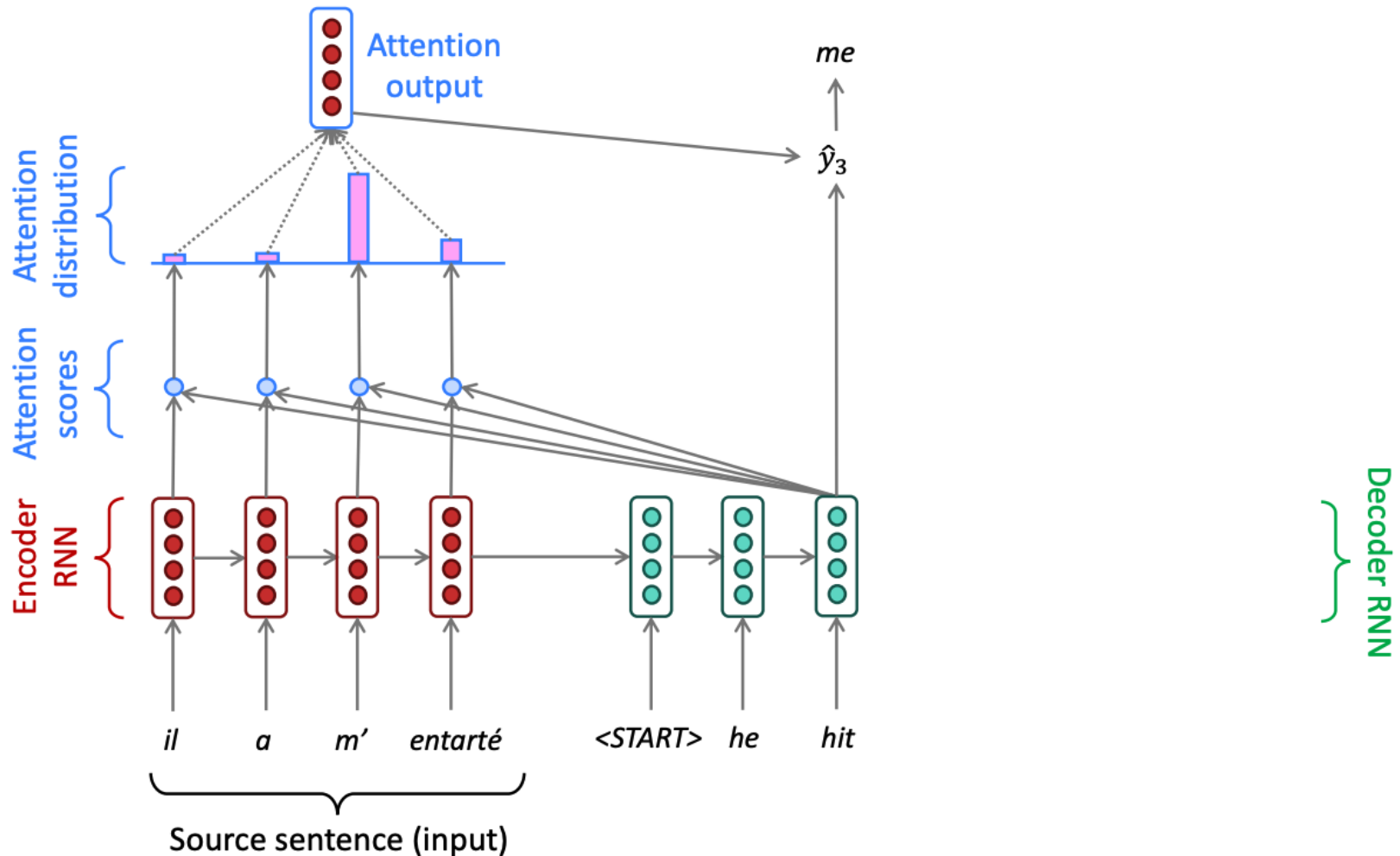
Attention



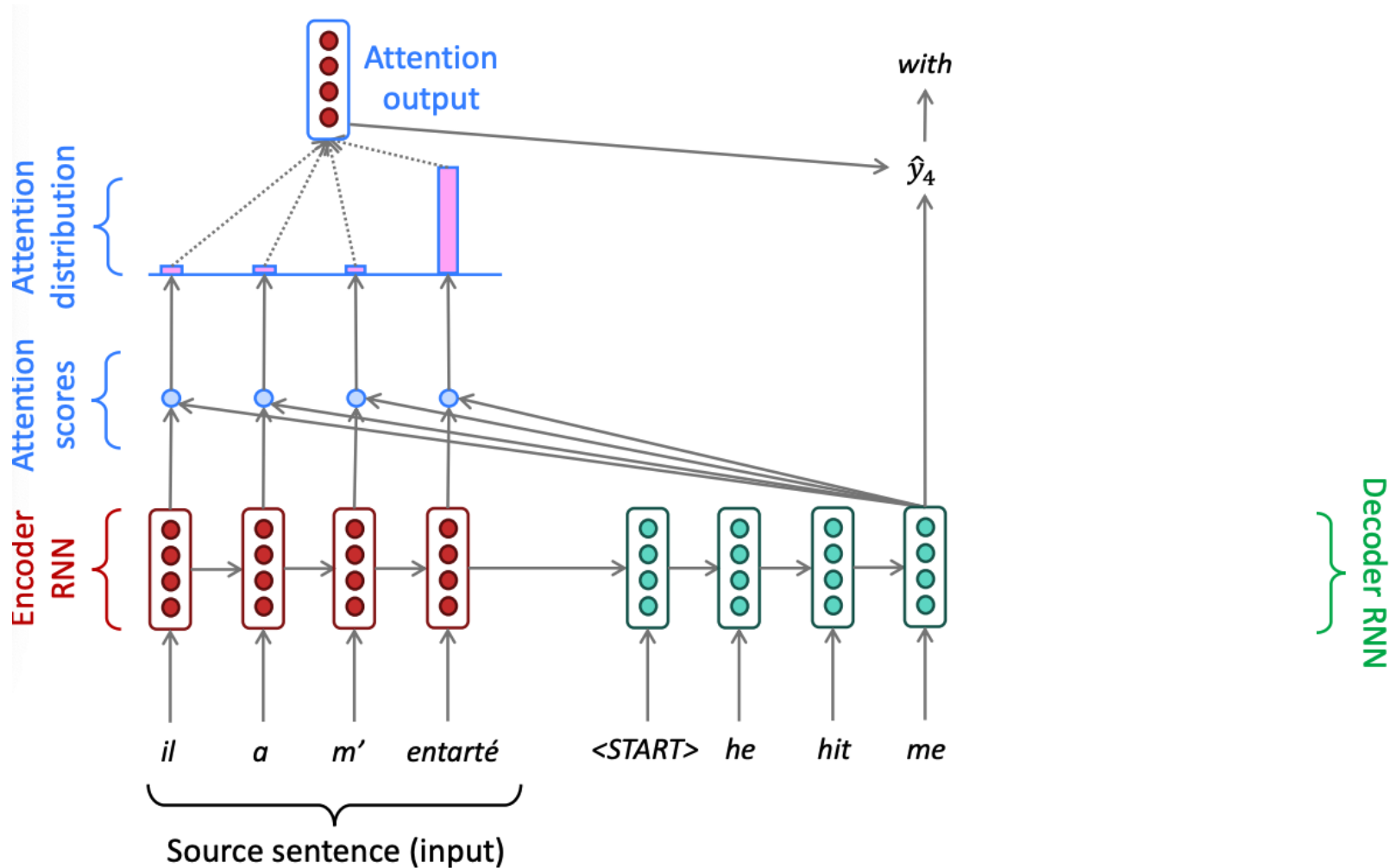
Attention



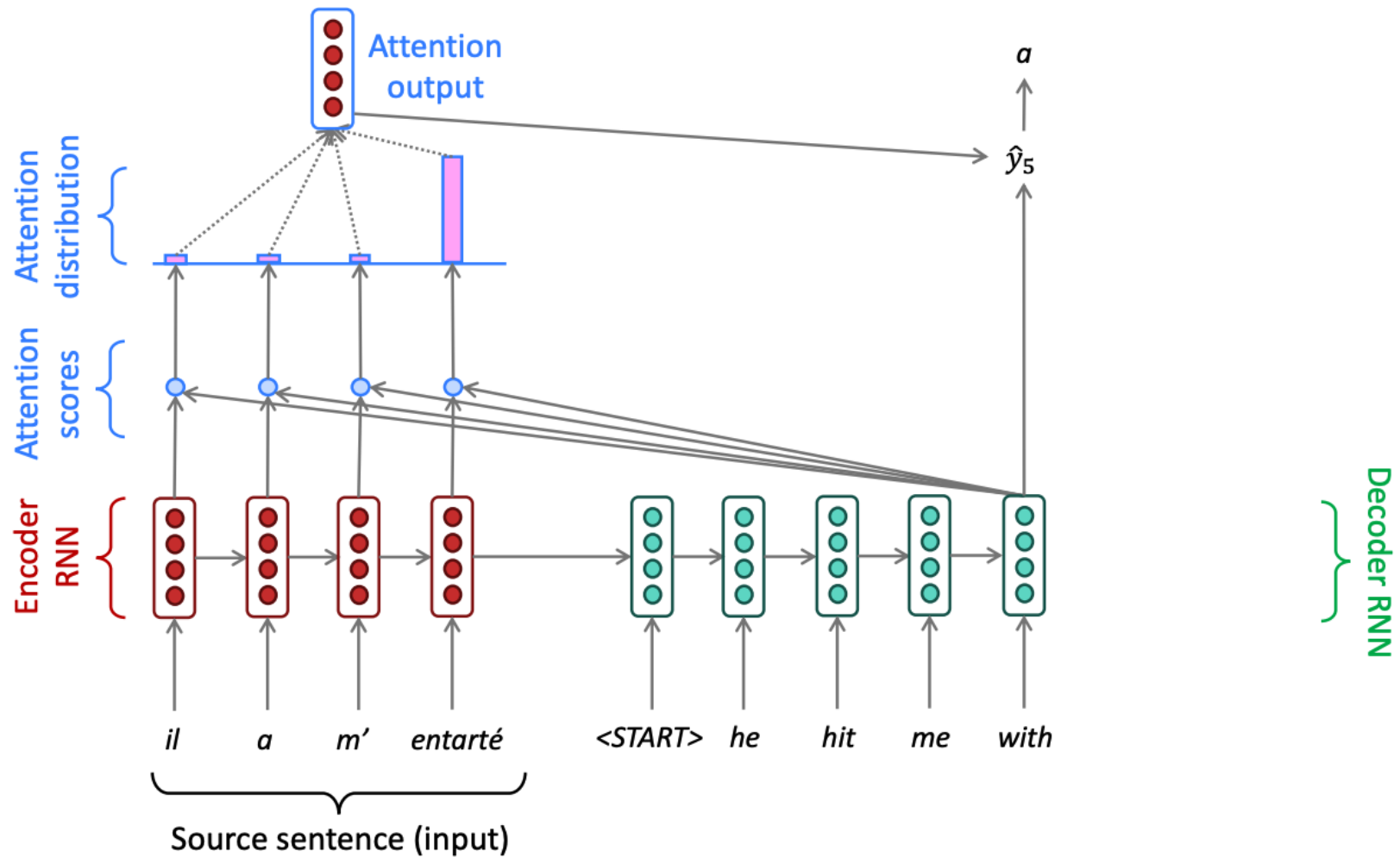
Attention



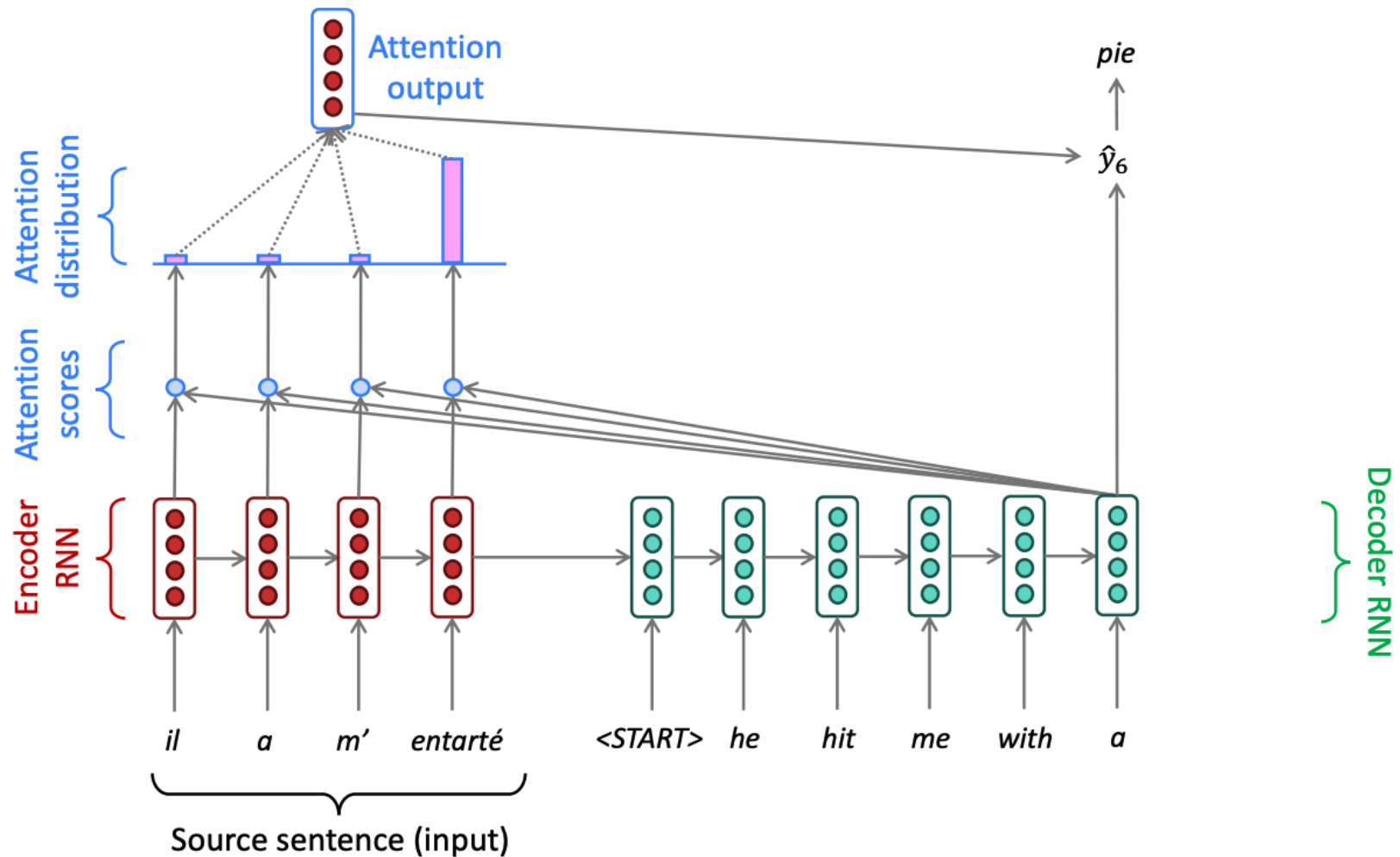
Attention



Attention



Attention



Attention

- Desta forma, em cada passo do decoder:
 - $score(h_t, s_k)$ -> Calculamos scores de atenção entre o estado do decoder e todos estados do encoder
 - Computamos uma softmax dos scores
 - Geramos o Attention Output como uma soma ponderada dos scores
- Attention Output é concatenado ao estado do decoder para a predição final
 - Desta forma, em cada passo do decoder:
 - $score(h_t, s_k)$ -> Calculamos scores de atenção entre o estado do decoder e todos estados do encoder
 - Computamos uma softmax dos scores
 - Geramos o Attention Output como uma soma ponderada dos scores

Encoder Hidden States

$s_1, s_2, s_3, \dots s_m$

Decoder Hidden State no
timestep t

h_t

Encoder Hidden States

$s_1, s_2, s_3, \dots s_m$

Decoder Hidden State no
timestep t

h_t

Quão relevante um token k é para o timestep t

$score(h_t, s_k), k = 1..m$

Encoder Hidden States

$s_1, s_2, s_3, \dots s_m$

Decoder Hidden State no
timestep t

h_t

Quão relevante um token k é para o timestep t

$score(h_t, s_k), k = 1..m$

Pesos de atenção para token k no timestep t

$$a_k^{(t)} = \frac{\exp(score(h_t, s_k))}{\sum_i^m \exp(score(h_t, s_i))}, k = 1..m$$

Encoder Hidden States

$s_1, s_2, s_3, \dots s_m$

Decoder Hidden State no
timestep t

h_t

Quão relevante um token k é para o timestep t

$score(h_t, s_k), k = 1..m$

Pesos de atenção para token k no timestep t

$$a_k^{(t)} = \frac{\exp(score(h_t, s_k))}{\sum_i^m \exp(score(h_t, s_i))}, k = 1..m$$

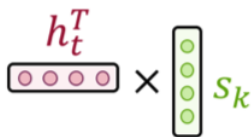
Contexto para o timestep t

$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \dots + a_m^{(t)} s_m = \sum_{k=1}^m a_k^{(t)} s_k$$

Attention

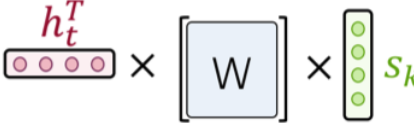
- Na prática, temos formas diferentes de calcular a atenção

Dot-product



$$\text{score}(h_t, s_k) = h_t^T s_k$$

Bilinear



$$\text{score}(h_t, s_k) = h_t^T W s_k$$

Luong Attention

Multi-Layer Perceptron



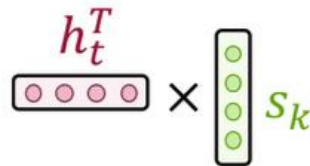
$$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1 [h_t, s_k])$$

Bahdanau Attention

Attention

- Temos ainda um problema:
 - Na versão que vimos de *Dot Product Attention* ainda sofremos de um antigo problema: **Vanishing Gradients**
 - O problema aparece quando aumentamos o tamanho da dimensão dos vetores Hidden

Dot-product



The diagram illustrates the dot product of two vectors. On the left, a horizontal vector h_t^T is shown with four pink circles inside a box. To its right is a vertical vector s_k with four green circles inside a box. A multiplication symbol $\times$ is placed between the two boxes.

$$\text{score}(h_t, s_k) = h_t^T s_k$$

Scaled Dot Product Attention

- Conforme aumentamos a dimensão d dos vetores a variância dos **scores** tende a d
- Dessa forma, ao calcular o *backpropagation*, os valores de scores seriam **muito altos**
- Para resolver o problema, podemos **normalizar** o dot product por $\sqrt{d}$, obtendo assim o Scaled Dot Product Attention

$$\text{score}(h_t, s_k) = \frac{h_t s_k}{\sqrt{d}}$$

Scaled Dot Product Attention

- Attention foi um grande mecanismo para as tarefas em NLP
- Embora tenha surgido para Machine Translation, podemos adaptá-la para diferentes arquiteturas

$$\text{score}(h_t, s_k) = \frac{h_t s_k}{\sqrt{d}}$$

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_i^m \exp(\text{score}(h_t, s_i))}, k = 1..m$$

$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \dots + a_m^{(t)} s_m = \sum_{k=1}^m a_k^{(t)} s_k$$

Scaled Dot Product Attention

- Mas e se tentássemos usar apenas Attention para aprender relações entre entradas?
- É possível?

$$\text{score}(h_t, s_k) = \frac{h_t s_k}{\sqrt{d}}$$

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_i^m \exp(\text{score}(h_t, s_i))}, k = 1..m$$

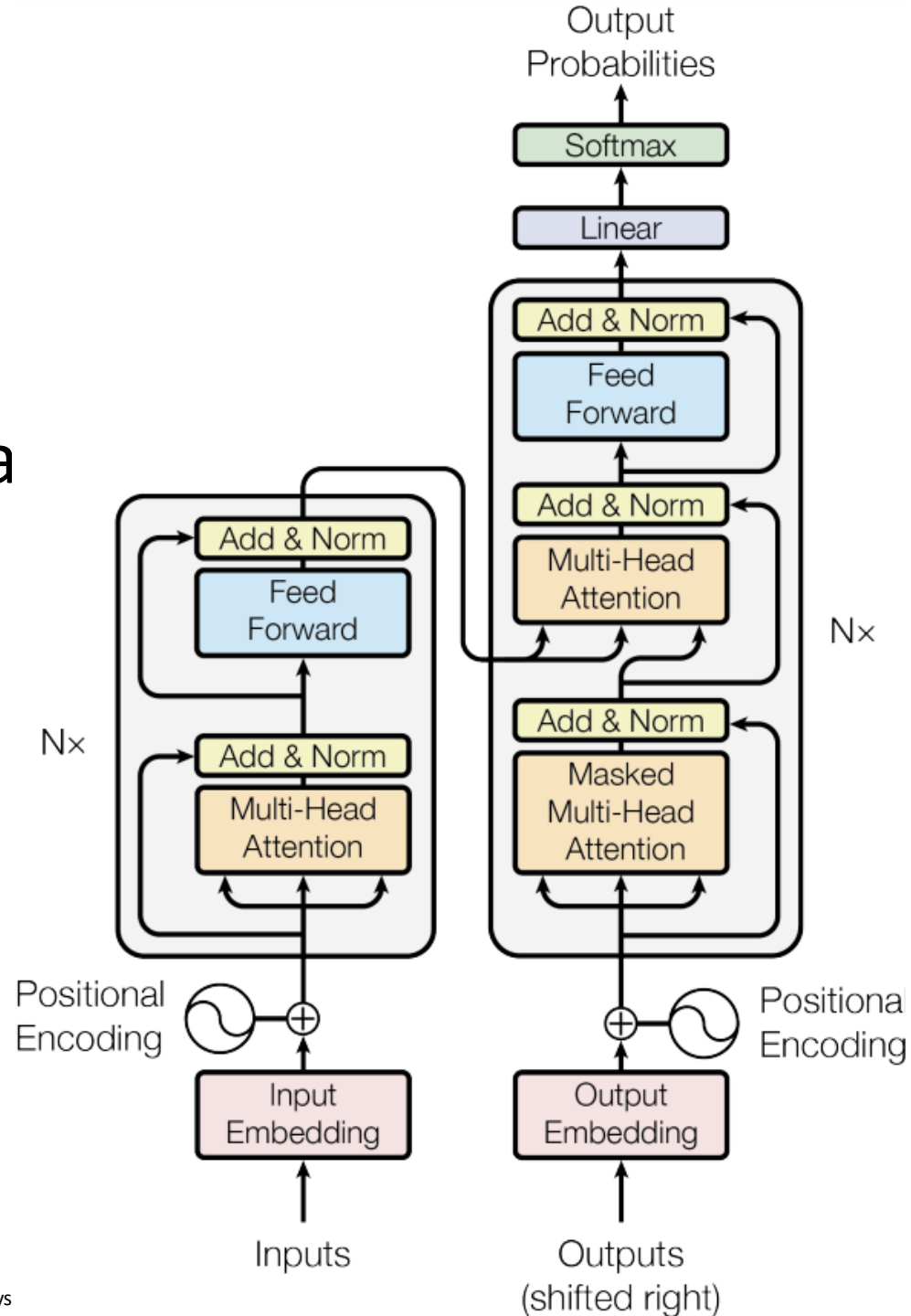
$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \dots + a_m^{(t)} s_m = \sum_{k=1}^m a_k^{(t)} s_k$$

Roteiro

- Transformers
 - Visão Geral
 - Self-Attention
 - Positional Encoding
 - Encoder
 - Decoder

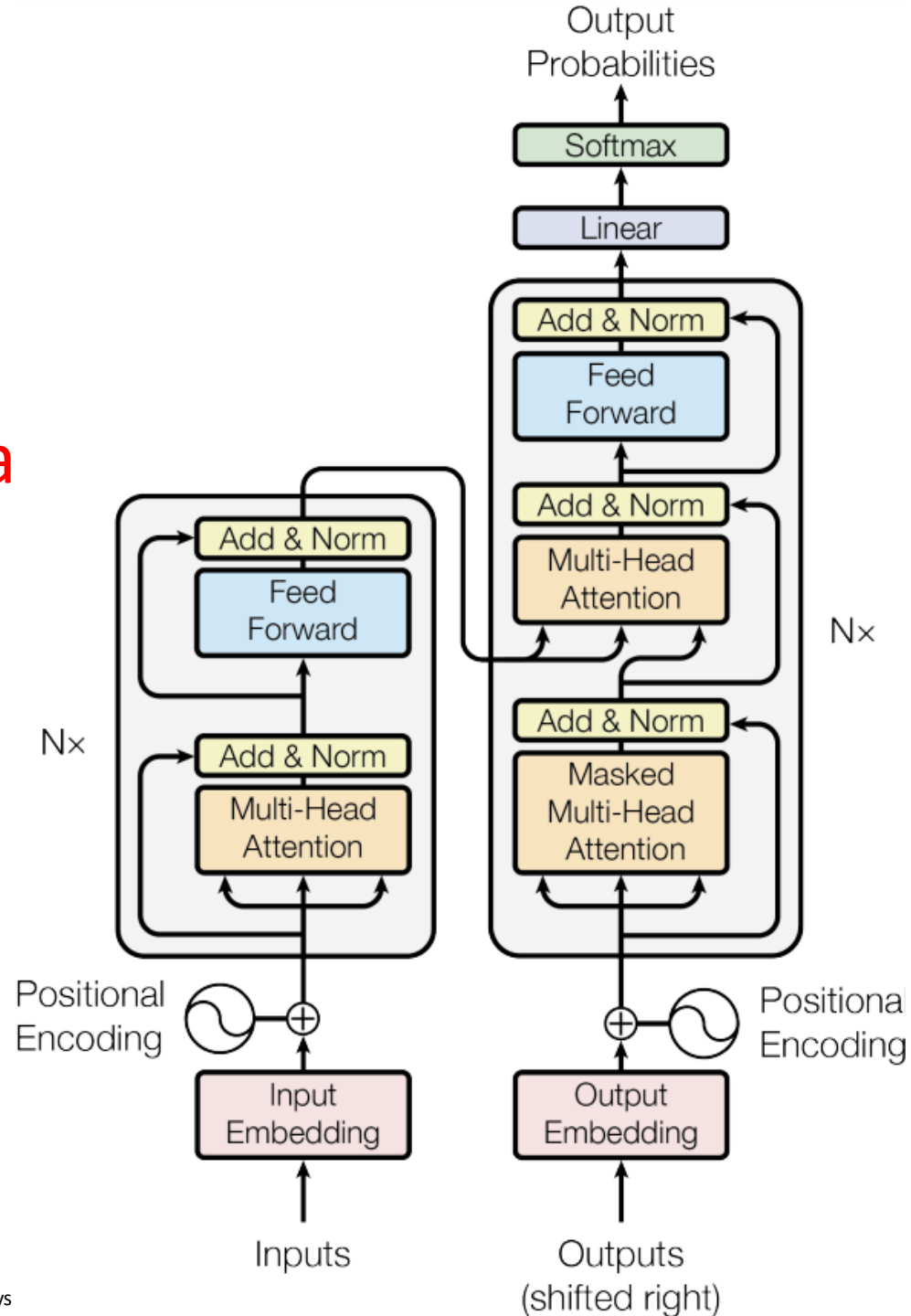
Transformers

- Arquitetura inteiramente baseada em atenção!



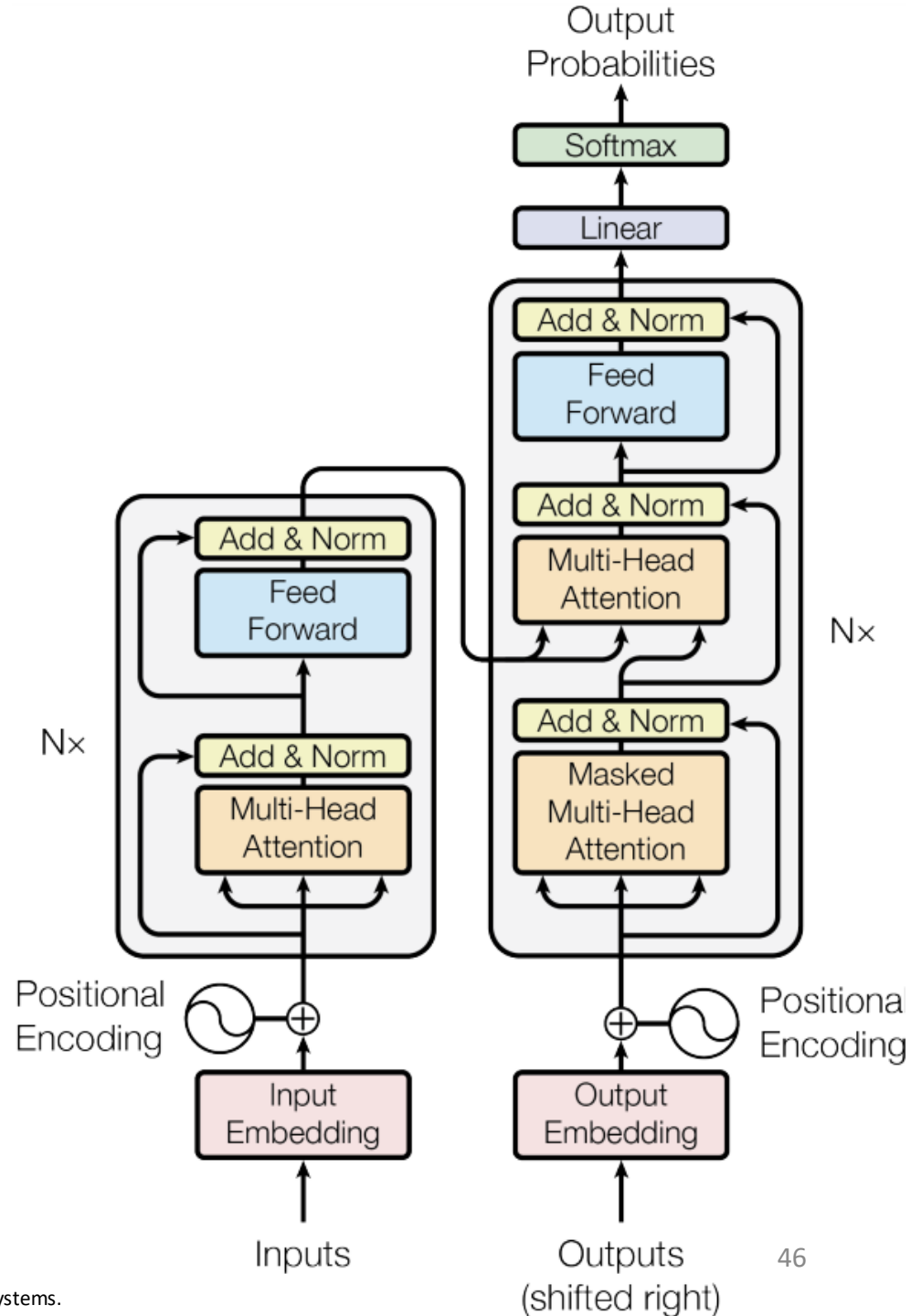
Transformers

- Nosso objetivo:
Tradução de Máquina



Transformers

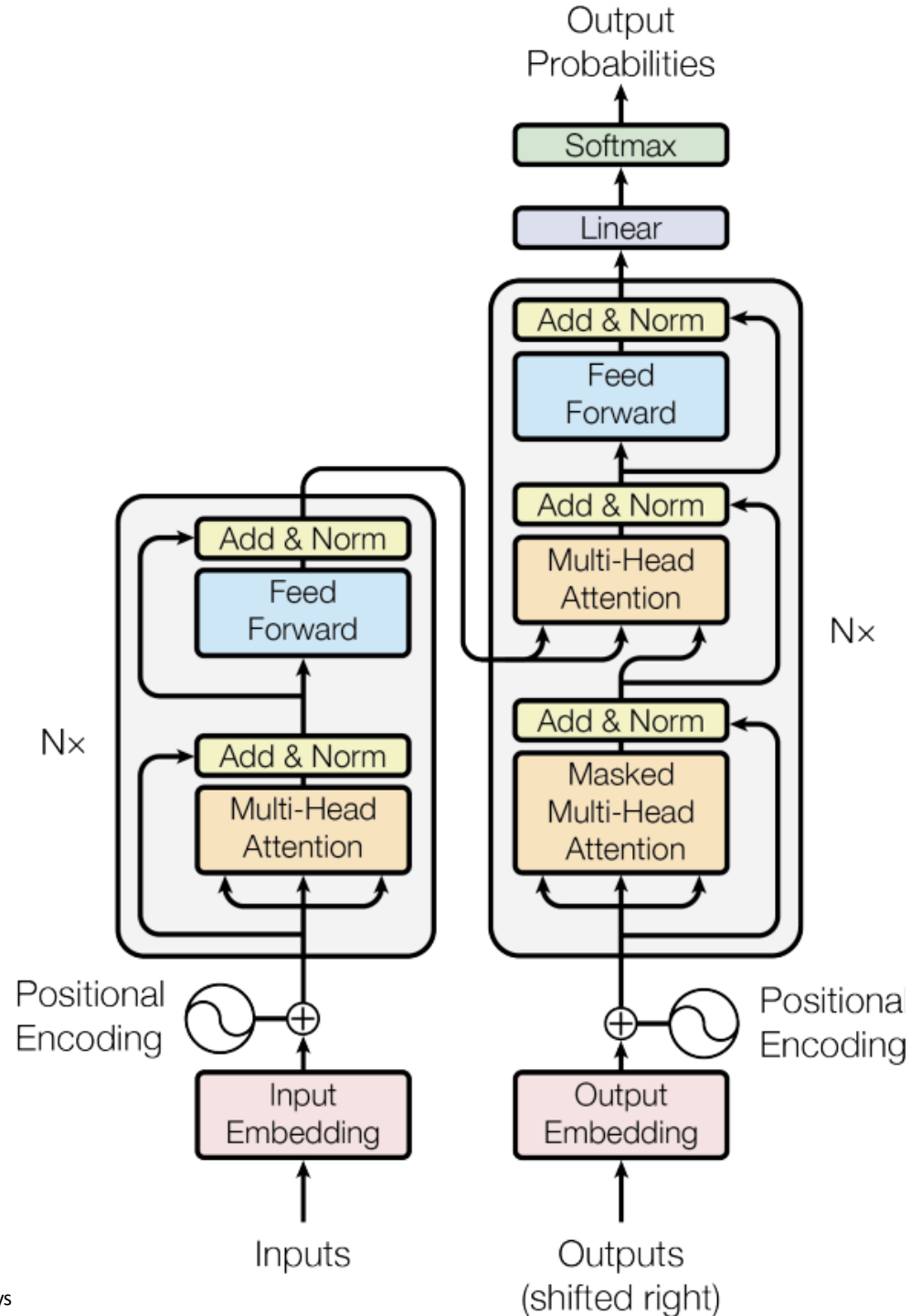
- Vamos encará-la por partes:
 - Self-Attention
 - Positional Encoding
 - Encoder
 - Decoder



Transformers

- Vamos encará-la por partes:

- Self-Attention
- Positional Encoding
- Encoder
- Decoder



Self-Attention

- Anteriormente havíamos visto o mecanismo de Scaled Dot Product Attention

$$\text{score}(h_t, s_k) = \frac{h_t s_k}{\sqrt{d}}$$

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_i^m \exp(\text{score}(h_t, s_i))}, k = 1..m$$

$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \dots + a_m^{(t)} s_m = \sum_{k=1}^m a_k^{(t)} s_k$$

Self-Attention

- Temos alguns problemas:
 - Como iremos usar apenas Self-Attention se nada na estrutura aprende?
 - Como tornar o mecanismo genérico?

$$\text{score}(h_t, s_k) = \frac{h_t s_k}{\sqrt{d}}$$

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_i^m \exp(\text{score}(h_t, s_i))}, k = 1..m$$

$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \dots + a_m^{(t)} s_m = \sum_{k=1}^m a_k^{(t)} s_k$$

Self-Attention

- Temos alguns problemas:
 - Como iremos usar apenas Self-Attention se nada na estrutura aprende?
 - Como tornar o mecanismo genérico?

$$\text{score}(h_t, s_k) = \frac{h_t s_k}{\sqrt{d}}$$

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_i^m \exp(\text{score}(h_t, s_i))}, k = 1..m$$

$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \cdots + a_m^{(t)} s_m = \sum_{k=1}^m a_k^{(t)} s_k$$

Self-Attention

- Como tornar o mecanismo genérico?
 - Vamos redefinir a atenção para trabalhar com qualquer par de vetores
 - Chamaremos eles de **Query** e **Key**

$$\text{score}(h_t, s_k) = \frac{h_t s_k}{\sqrt{d}}$$

$$\text{score}(Q, K) = \frac{Q \cdot K^T}{\sqrt{d}}$$

Self-Attention

- Como tornar o mecanismo genérico?
 - Vamos redefinir a atenção para trabalhar com qualquer par de vetores
 - Chamaremos eles de **Query** e **Key**
 - Após calcular os scores, aplicávamos uma **softmax**

$$attention(Q, K) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right)$$

Self-Attention

- Como tornar o mecanismo genérico?
 - Vamos redefinir a atenção para trabalhar com qualquer par de vetores
 - No final tínhamos uma soma ponderada com os valores originais dos estados, vamos resumir apenas adicionando um termo, **Value**

$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \dots + a_m^{(t)} s_m = \sum_{k=1}^m a_k^{(t)} s_k$$

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

- Como tornar o mecanismo genérico?
 - Vamos redefinir a atenção para trabalhar com qualquer par de vetores
 - Mapeamos assim todas as aparições dos vetores originais para novos vetores

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

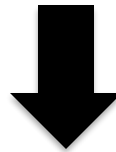
Self-Attention

- Como tornar o mecanismo genérico?

$$\text{score}(h_t, s_k) = \frac{h_t s_k}{\sqrt{d}}$$

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_i^m \exp(\text{score}(h_t, s_i))}, k = 1..m$$

$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \dots + a_m^{(t)} s_m = \sum_{k=1}^m a_k^{(t)} s_k$$



$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

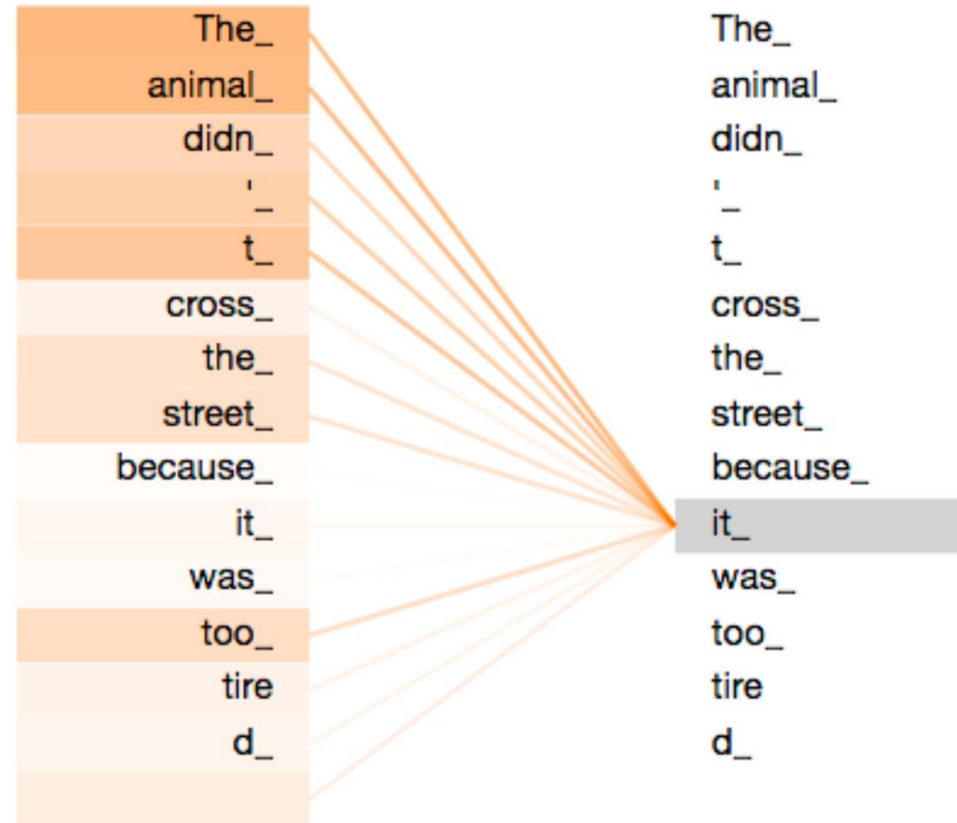
Self-Attention

- Como tornar o mecanismo genérico?
 - Podemos agora dizer que: Cada token de entrada faz uma **Query** contra as **Keys** de outros tokens para retornar um **Value** específico conforme uma soma ponderada da similaridade da **Query** com as **Keys**

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

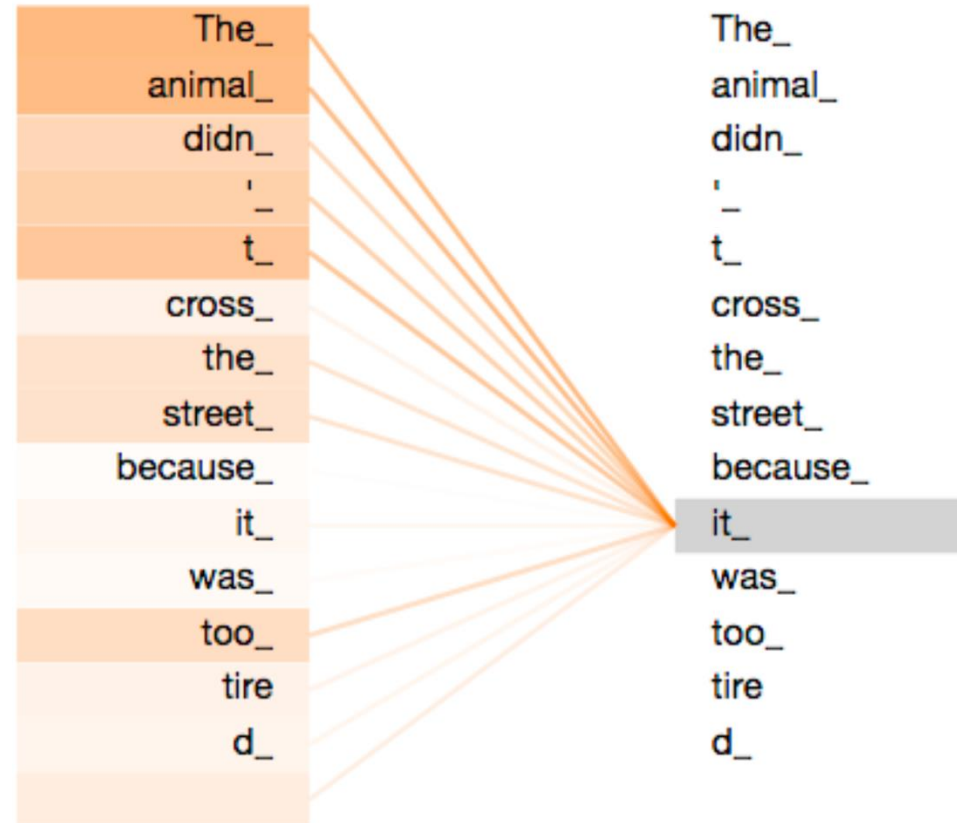
- Como tornar o mecanismo genérico?
 - Desta forma, poderemos relacionar cada token com os demais tokens de uma mesma sentença



$$attention(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

- Como tornar o mecanismo genérico?
 - Desta forma, poderemos relacionar cada token com os demais tokens de uma mesma sentença
 - Mas como exatamente?



$$attention(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

- Como tornar o mecanismo genérico?
 - Em um primeiro momento poderíamos usar Q , K e V como os embeddings dos tokens na sentença:
 - Q : O embedding de uma palavra específica
 - K : Os embeddings das demais palavras
 - V : Os embeddings das demais palavras

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

- Temos alguns problemas:
 - Como iremos usar apenas Self-Attention se nada na estrutura aprende?
 - Como tornar o mecanismo genérico?
 - Resolvido! $attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$

Self-Attention

- Como iremos usar apenas Self-Attention se nada na estrutura aprende?
 - Nada na atual composição nos permite aprender algo. Como resolver?

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

- Como iremos usar apenas Self-Attention se nada na estrutura aprende?
 - Nada na atual composição nos permite aprender algo. Como resolver?

$$Q = x \cdot W_q + b_q$$

$$K = x \cdot W_k + b_k$$

$$V = x \cdot W_v + b_v$$

Adicionamos uma camada linear para cada uma das entradas

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

- Como iremos usar apenas Self-Attention se nada na estrutura aprende?
 - Nada na atual composição nos permite aprender algo. Como resolver?

$$Q = x \cdot W_q + b_q$$

$$K = x \cdot W_k + b_k$$

$$V = x \cdot W_v + b_v$$

Mesmo
embedding do
token

$$attention(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

- Como iremos usar apenas Self-Attention se nada na estrutura aprende?
 - Nada na atual composição nos permite aprender algo. Como resolver?

$$attention(x \cdot W_q + b_q, x \cdot W_k + b_k, x \cdot W_v + b_v) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

- Temos alguns problemas:
 - Como iremos usar apenas Self-Attention se nada na estrutura aprende?
 - Resolvido! $Q = x \cdot W_q + b_q$
 $K = x \cdot W_k + b_k$
 $V = x \cdot W_v + b_v$
 - Como tornar o mecanismo genérico?
 - Resolvido! $attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$

Self-Attention

- Vamos ver um exemplo!

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

W_q

2	1
3	1
1	2

W_k

1	1
2	3
1	1

W_v

1	3
2	1
4	1

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

Queries

-1	3
----	---

W_q

2	1
3	1
1	2

W_k

1	1
2	3
1	1

W_v

1	3
2	1
4	1

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

Queries

-1	3
----	---

-5	1
----	---

W_q

2	1
3	1
1	2

W_k

1	1
2	3
1	1

W_v

1	3
2	1
4	1

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

Queries

-1	3
----	---

-5	1
----	---

Keys

2	4
---	---

-2	-3
----	----

W_q

2	1
3	1
1	2

W_k

1	1
2	3
1	1

W_v

1	3
2	1
4	1

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

W_q

2	1
3	1
1	2

Queries

-1	3
----	---

-5	1
----	---

Keys

2	4
---	---

-2	-3
----	----

W_k

1	1
2	3
1	1

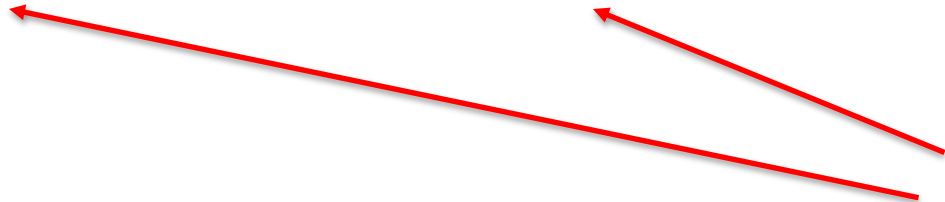
Values

11	-10
----	-----

4	5
---	---

W_v

1	3
2	1
4	1



$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

Queries

-1	3
----	---

-5	1
----	---

Keys

2	4
---	---

-2	-3
----	----

Values

11	-10
----	-----

4	5
---	---

W_q

2	1
3	1
1	2

W_k

1	1
2	3
1	1

W_v

1	3
2	1
4	1

Estes vetores são
os que entram no
nosso módulo de
Self-Attention!

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

Queries

-1	3
----	---

-5	1
----	---

Keys

2	4
---	---

-2	-3
----	----

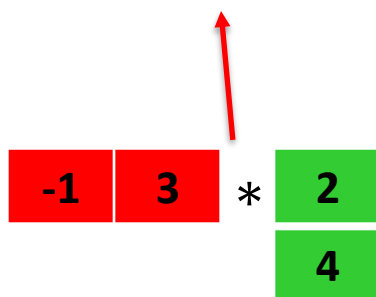
Values

11	-10
----	-----

4	5
---	---

Scores

10



W_q

2	1
3	1
1	2

W_k

1	1
2	3
1	1

W_v

1	3
2	1
4	1

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

Queries

-1	3
----	---

-5	1
----	---

Keys

2	4
---	---

-2	-3
----	----

Values

11	-10
----	-----

4	5
---	---

Scores

10

-7

-5	1	*	-2
			-3

W_q

2	1
3	1
1	2

W_k

1	1
2	3
1	1

W_v

1	3
2	1
4	1

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

W_q

2	1
3	1
1	2

Queries

-1	3
----	---

-5	1
----	---

Keys

2	4
---	---

-2	-3
----	----

W_k

1	1
2	3
1	1

Values

11	-10
----	-----

4	5
---	---

Scores

10

-7

Normalização

$$10 \cdot \frac{1}{\sqrt{d}} = 7.07$$

$$-7 \cdot \frac{1}{\sqrt{d}} = -4.95$$

W_v

1	3
2	1
4	1

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

W_q

2	1
3	1
1	2

Queries

-1	3
----	---

-5	1
----	---

Keys

2	4
---	---

-2	-3
----	----

W_k

1	1
2	3
1	1

Values

11	-10
----	-----

4	5
---	---

Scores

10

-7

Normalização

$$10 \cdot \frac{1}{\sqrt{d}} = 7.07$$

$$-7 \cdot \frac{1}{\sqrt{d}} = -4.95$$

W_v

1	3
2	1
4	1

Softmax

0.99

0.01

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

$$W_q$$

2	1
3	1
1	2

Queries

-1	3
----	---

-5	1
----	---

Keys

2	4
---	---

-2	-3
----	----

$$W_k$$

1	1
2	3
1	1

Values

11	-10
----	-----

4	5
---	---

Scores

10

-7

Normalização

$$10 \cdot \frac{1}{\sqrt{d}} = 7.07$$

$$-7 \cdot \frac{1}{\sqrt{d}} = -4.95$$

$$W_v$$

1	3
2	1
4	1

Softmax

0.99

0.01

Soma Ponderada

11	-10	*	0.99
----	-----	---	------

4	5	*	0.01
---	---	---	------

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

$$W_q$$

2	1
3	1
1	2

Queries

-1	3
----	---

-5	1
----	---

Keys

2	4
---	---

-2	-3
----	----

$$W_k$$

1	1
2	3
1	1

Values

11	-10
----	-----

4	5
---	---

Scores

10

-7

Normalização

$$10 \cdot \frac{1}{\sqrt{d}} = 7.07$$

$$-7 \cdot \frac{1}{\sqrt{d}} = -4.95$$

$$W_v$$

1	3
2	1
4	1

Softmax

0.99

0.01

Soma Ponderada

11	-10	*	0.99
----	-----	---	------

4	5	*	0.01
---	---	---	------

10.93	-9.95
-------	-------

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Embeddings

-5	2	3
----	---	---

-2	-1	2
----	----	---

$$W_q$$

2	1
3	1
1	2

Queries

-1	3
----	---

-5	1
----	---

$$W_k$$

1	1
2	3
1	1

Keys

2	4
---	---

-2	-3
----	----

Values

11	-10
----	-----

4	5
---	---

$$W_v$$

1	3
2	1
4	1

Scores

10

-7

Normalização

Este resultado nos indica como o primeiro token se relaciona com ele mesmo e com o segundo token da sentença!

Softmax

0.99

0.01

Soma Ponderada

11	-10	*	0.99
----	-----	---	------

4	5	*	0.01
---	---	---	------

10.93 -9.95

Self-Attention

- Na prática, o processo que vimos é feito de forma inteiramente matricial!

$$Q = x \cdot W_q + b_q$$

seq_size x emb_size emb_size x d

$$K = x \cdot W_k + b_k$$

seq_size x emb_size emb_size x d

$$V = x \cdot W_v + b_v$$

seq_size x emb_size emb_size x value_size

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

- Na prática, o processo que vimos é feito de forma inteiramente matricial!

seq_size x d d x seq_size

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

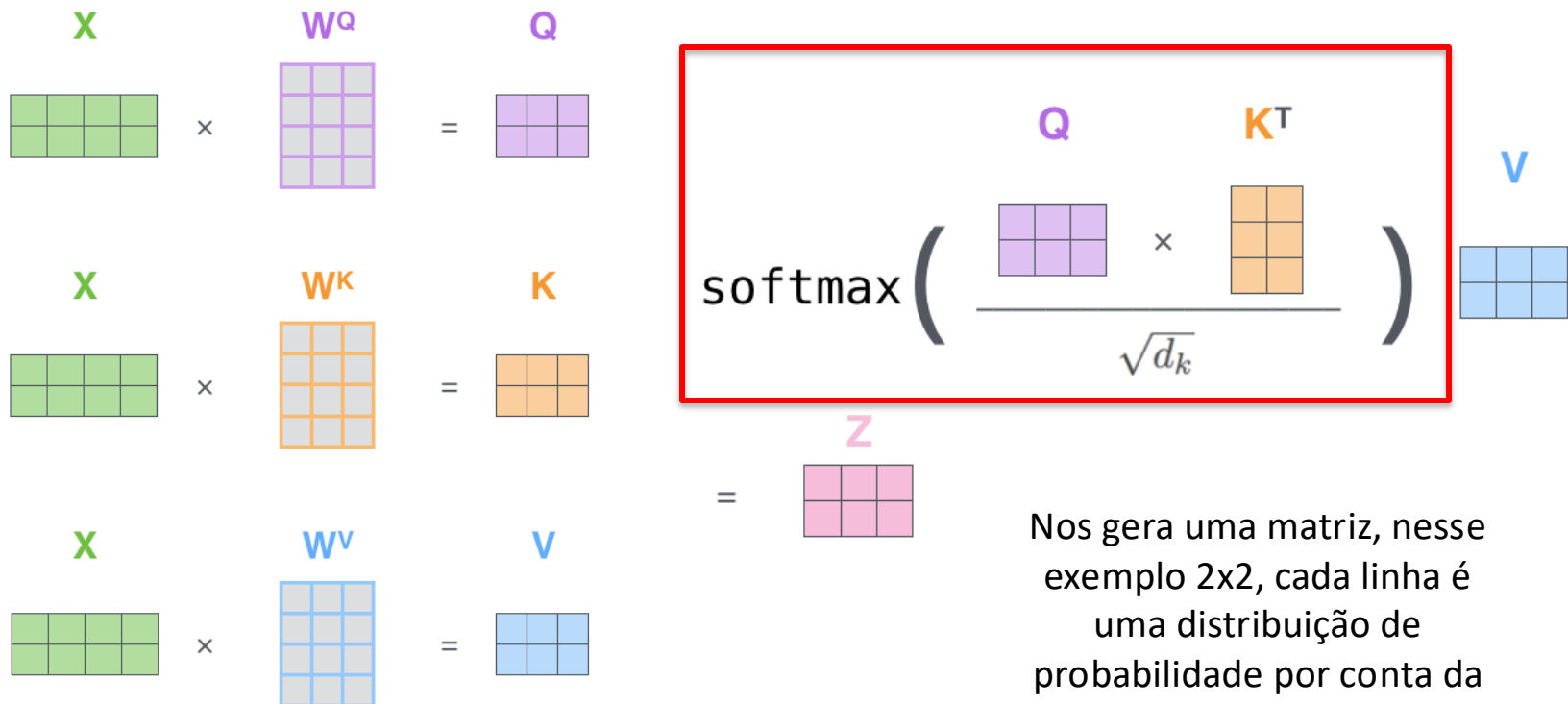
- Na prática, o processo que vimos é feito de forma inteiramente matricial!

seq_size x seq_size seq_size x value_size

$$attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

Self-Attention

- Na prática, o processo que vimos é feito de forma inteiramente matricial!



Nos gera uma matriz, nesse exemplo 2x2, cada linha é uma distribuição de probabilidade por conta da softmax!!!

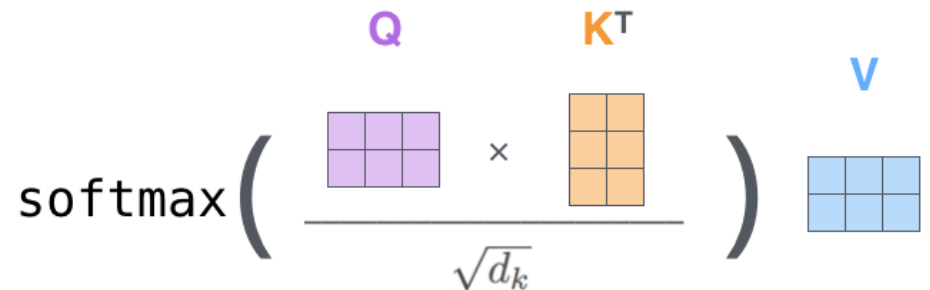
Self-Attention

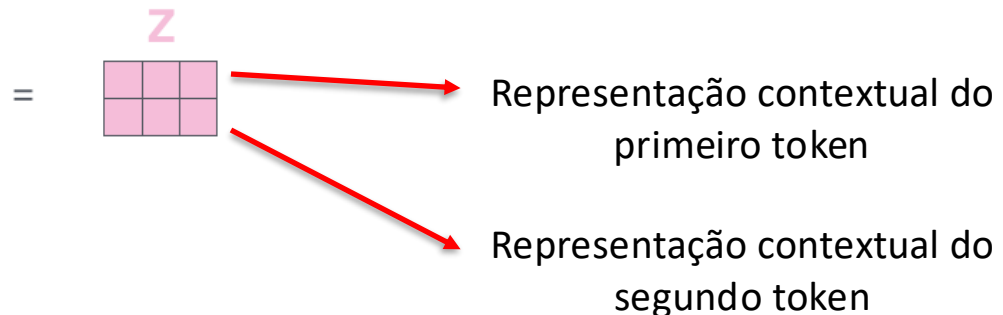
- Na prática, o processo que vimos é feito de forma inteiramente matricial!

$$X \times W^Q = Q$$


$$X \times W^K = K$$


$$X \times W^V = V$$


$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) \times V$$


$$= Z$$


Representação contextual do primeiro token

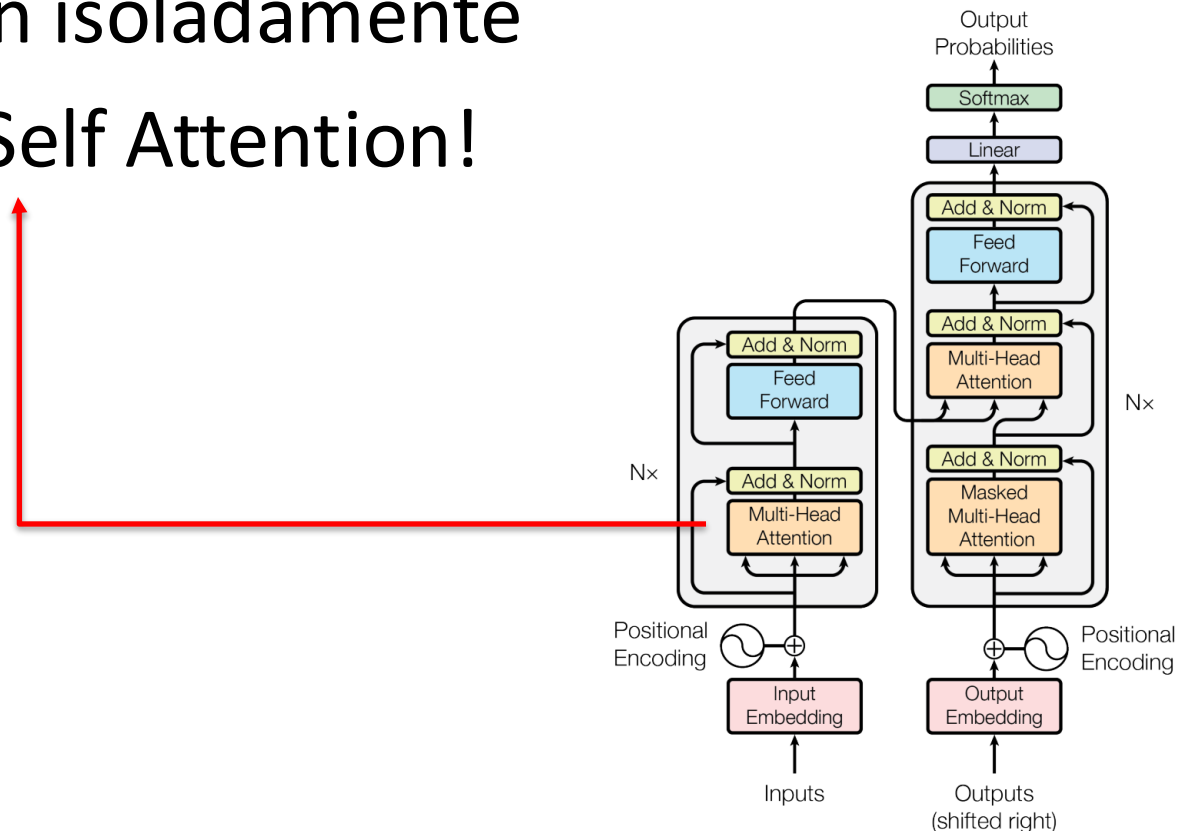
Representação contextual do segundo token

Self-Attention

- Alguns pontos relevantes sobre Self-Attention:
 - É não linear, além das operações de multiplicação ainda temos a softmax
 - Vetores de parâmetros não escalam com o tamanho da sequência de entrada
 - Facilmente paralelizável
 - Lida bem com sequências longas
 - Temos interpretabilidade: podemos visualizar os pesos da atenção (saída da softmax!)

Self-Attention

- Todavia, na arquitetura original, não usamos self-attention isoladamente
- Multi-Head Self Attention!



Self-Attention

- Calculamos o Self-Attention com várias matrizes diferentes

Attention Head #0

Diagram illustrating the input matrices for Attention Head #0. The input matrix (red) is a 3x2 grid. The key matrix (W_k , green) is a 3x2 grid. The value matrix (W_v , magenta) is a 3x2 grid.

		1	3
	1	1	1
2	1	3	1
3	1	1	
1	2		

Attention Head #1

Diagram illustrating the input matrices for Attention Head #1. The input matrix (red) is a 3x2 grid. The key matrix (W_k , green) is a 3x2 grid. The value matrix (W_v , magenta) is a 3x2 grid.

		1	3
	1	2	4
1	7	1	11
3	-2	-2	
1	5		

Attention Head #2

Diagram illustrating the input matrices for Attention Head #2. The input matrix (red) is a 3x2 grid. The key matrix (W_k , green) is a 3x2 grid. The value matrix (W_v , magenta) is a 3x2 grid.

		6	1
	12	2	0
6	2	2	4
3	5	0	
1	4		

Self-Attention

- Calculamos o Self-Attention com várias matrizes diferentes

Attention Head #0 Attention Head #1 Attention Head #2

10.93	-9.95
12.45	5.34

5.67	-1.23
1.09	0.87

9.23	1.68
9.33	-8.92

Para o output, concatenamos as
matrizes resultantes

10.93	-9.95	5.67	-1.23	9.23	1.68
12.45	5.34	1.09	0.87	9.33	-8.92

Self-Attention

- Calculamos o Self-Attention com várias matrizes diferentes

Para o output, concatenamos as matrizes resultantes

10.93	-9.95	5.67	-1.23	9.23	1.68
12.45	5.34	1.09	0.87	9.33	-8.92

Ao final, multiplicamos essa concatenação por uma matriz

W_o



Self-Attention

- Calculamos o Self-Attention com várias matrizes diferentes

Para o output, concatenamos as matrizes resultantes

10.93	-9.95	5.67	-1.23	9.23	1.68
12.45	5.34	1.09	0.87	9.33	-8.92

W_o

...	...	...
...	...	...

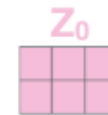
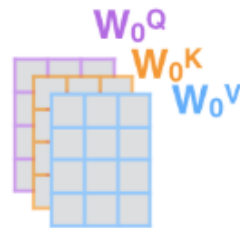
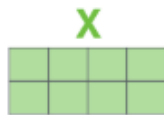
Representação contextual do primeiro token baseada em todas heads

Representação contextual do segundo token baseada em todas heads

Self-Attention

- Calculamos o Self-Attention com várias matrizes diferentes

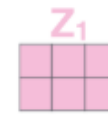
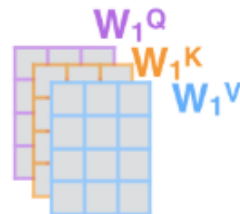
Thinking
Machines



W^O



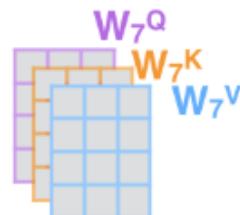
* In all encoders other than #0,
we don't need embedding.
We start directly with the output
of the encoder right below this one



...

...

...



Self-Attention

- Desta forma, criamos **vários sub-espacos** para poder representar uma sentença
 - Podemos tratar como: “**Diferentes** formas de **compreender e interpretar** uma sentença.”
- Para cada Attention Head, temos diferentes matrizes de W_q , W_k , W_v e apenas uma matriz compartilhada W_o
- Embeddings também são compartilhados entre todas heads

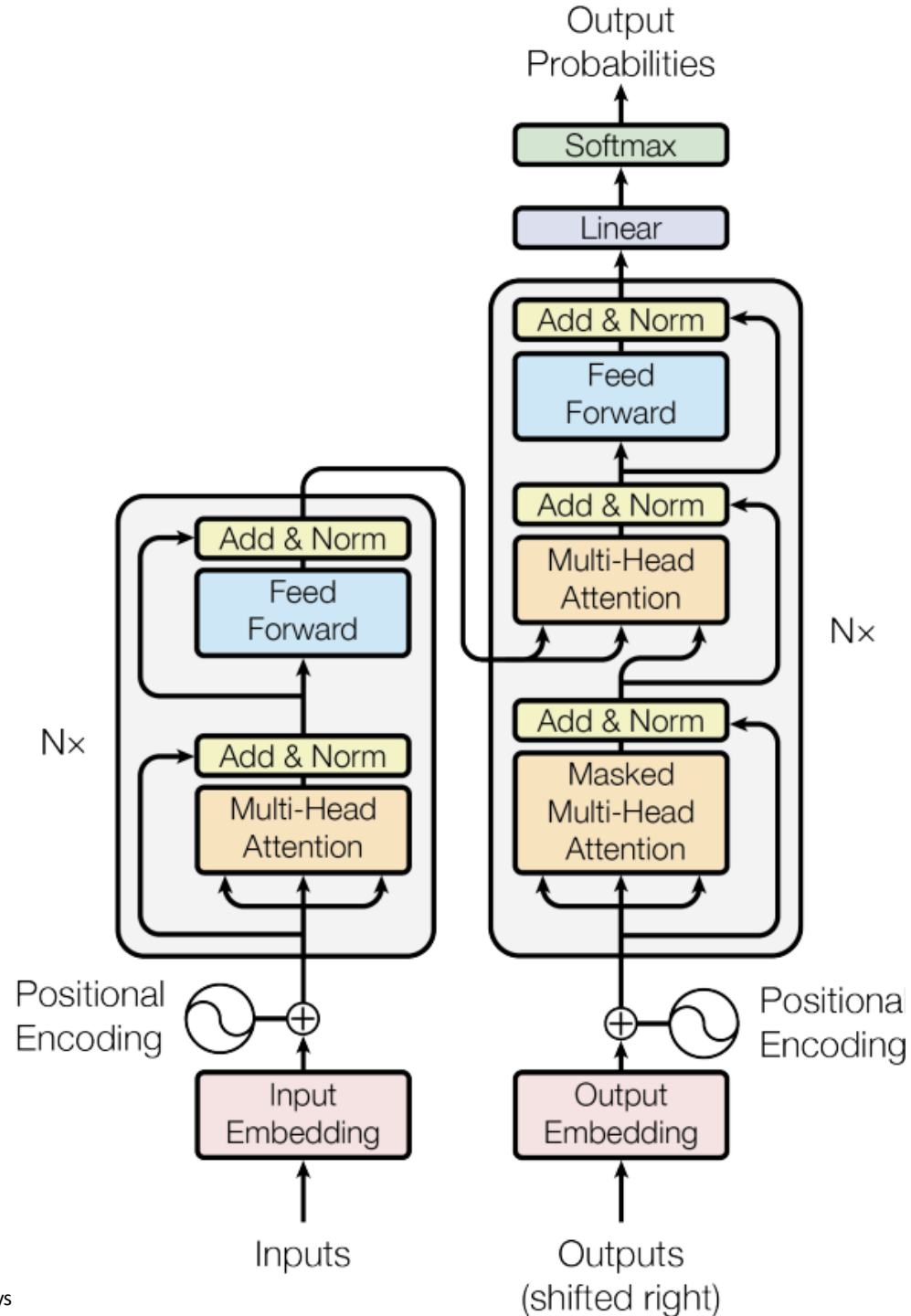
Self-Attention

- Podemos compreender os valores de **Query**, **Key** e **Value** como abstrações para:
 - Query: Simboliza o que uma determinada **palavra procura**
 - Ex: Um substantivo pode procurar por adjetivos
 - Key: Simboliza o que uma determinada **palavra tem a oferecer**
 - Ex: se o token é um verbo ou adjetivo
 - Value: Simboliza as **informações** que queremos **passar adiante**
 - Qual a relação de fato entre os tokens

Transformers

- Vamos encararla por partes:

- Self-Attention
- Positional Encoding
- Encoder
- Decoder



Positional Encoding

- Até agora vimos como processar embeddings
- Mas há um problema que não discutimos até agora
- Nas RNNs, nossa principal camada era **sequencial** e tínhamos um *Hidden state*
 - Ou seja, naturalmente conseguíamos lidar com **temporalidade**!
- E agora, onde está a informação temporal?

Positional Encoding

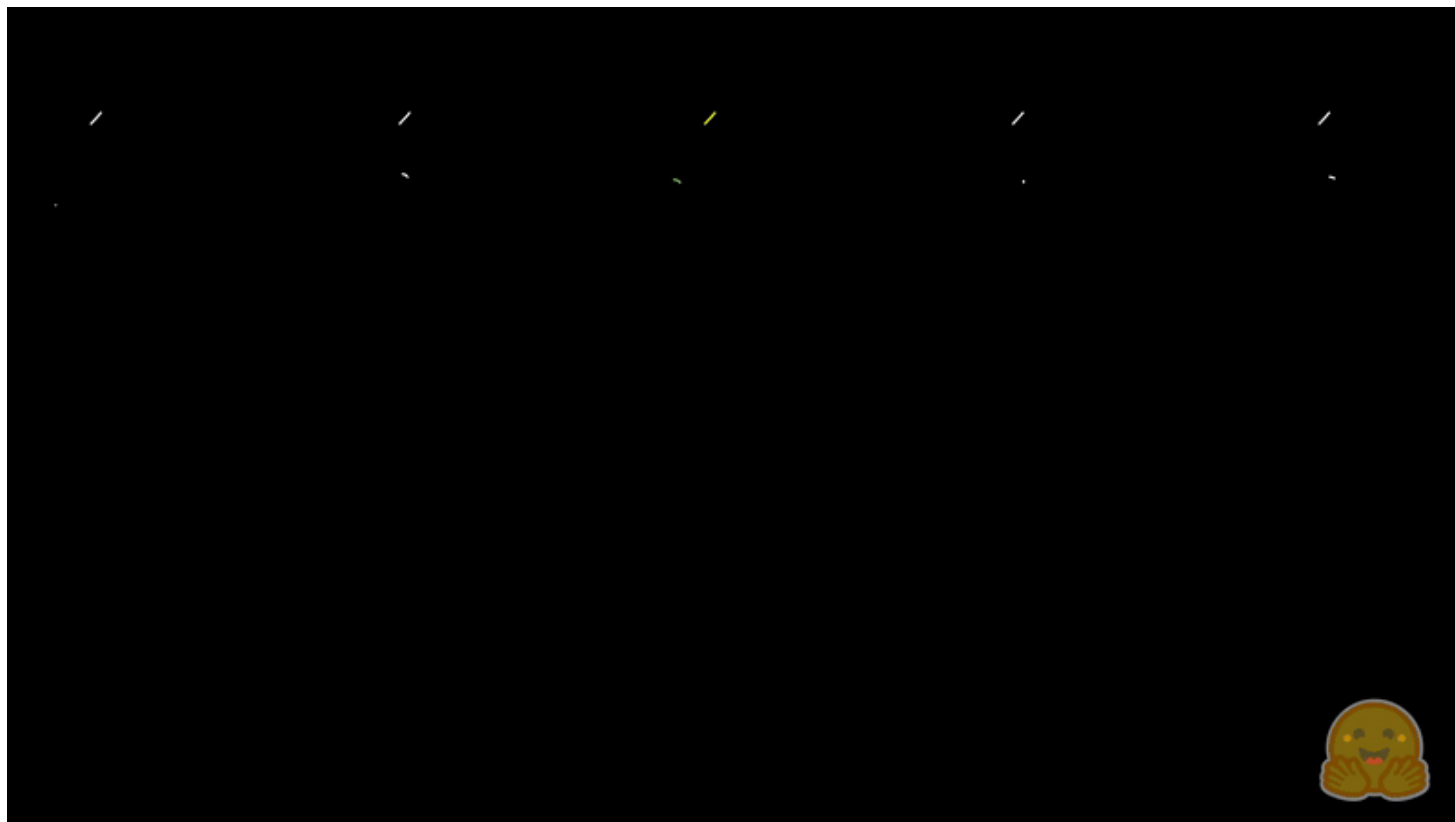
- Até agora vimos como processar embeddings
- Mas há um problema que não discutimos até agora
- Nas RNNs, nossa principal camada era **sequencial** e tínhamos um *Hidden state*
 - Ou seja, naturalmente conseguíamos lidar com **temporalidade**!
- E agora, onde está a informação temporal?
 - Não Está 😞

Positional Encoding

- Podemos adicionar a temporalidade no próprio embedding
- Gostaríamos de algumas propriedades:
 - 1: Codificar cada posição de **maneira única**
 - Cada token ocupa uma única posição e cada posição é ocupada por um único token
 - 2: Ter uma **relação linear entre duas posições**
 - Permitir o modelo compreender as posições de forma mais fácil -> Ex: distância de 10 para 5 ou 15 é 5
 - 3: Ser gerada por um **processo determinístico**
 - Permitir o modelo a aprender facilmente como definimos essa informação

Positional Encoding

- Alternativas? Valor fixo?

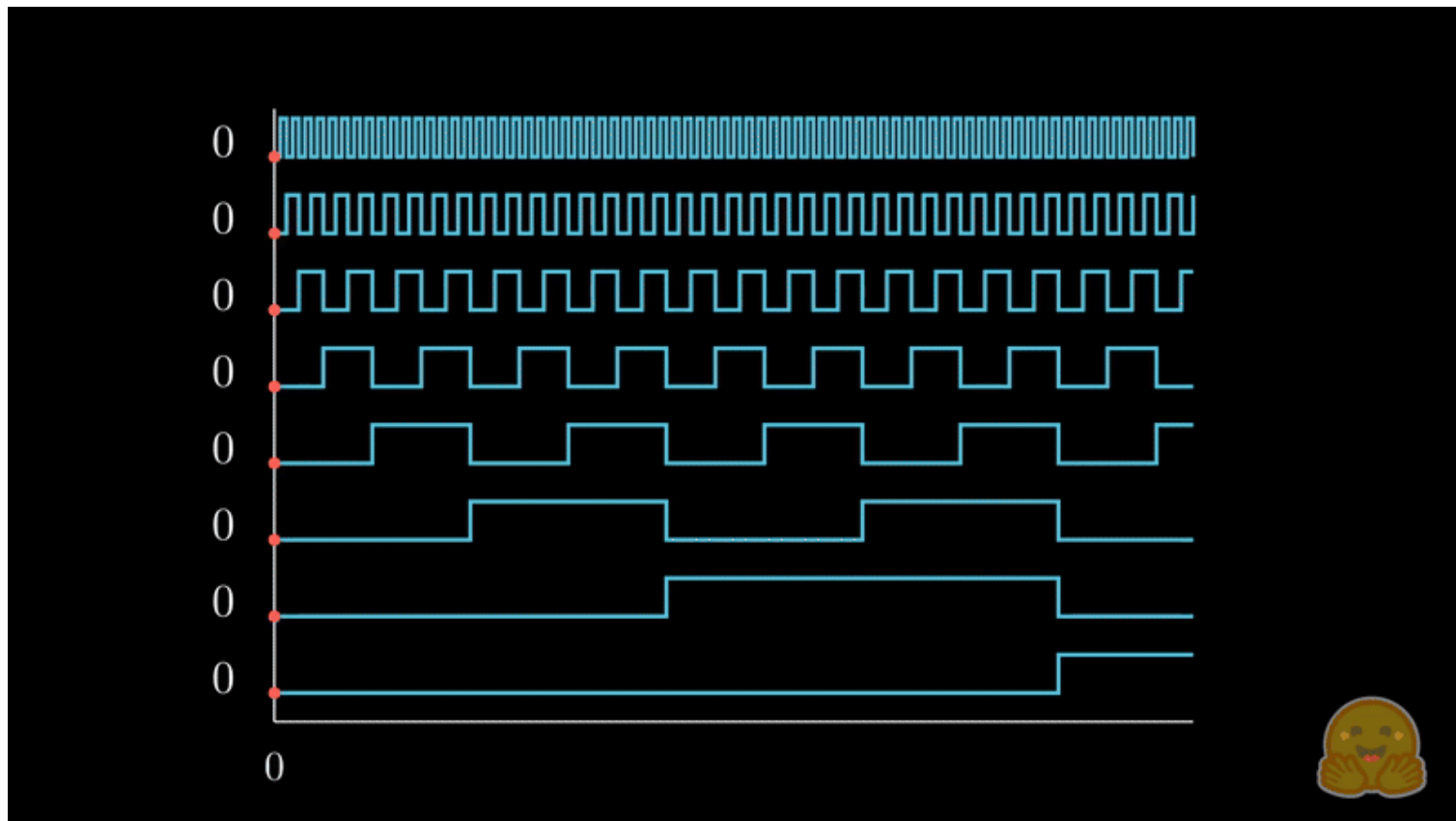


Positional Encoding

- Valores fixos:
 - Não temos um limite superior
 - Valores podem se tornar muito altos em relação aos valores originais dos embeddings (média=0)
 - Poderíamos normalizar pela quantidade de tokens, mas mudaria para cada entrada

Positional Encoding

- Alternativas? Valores binários?

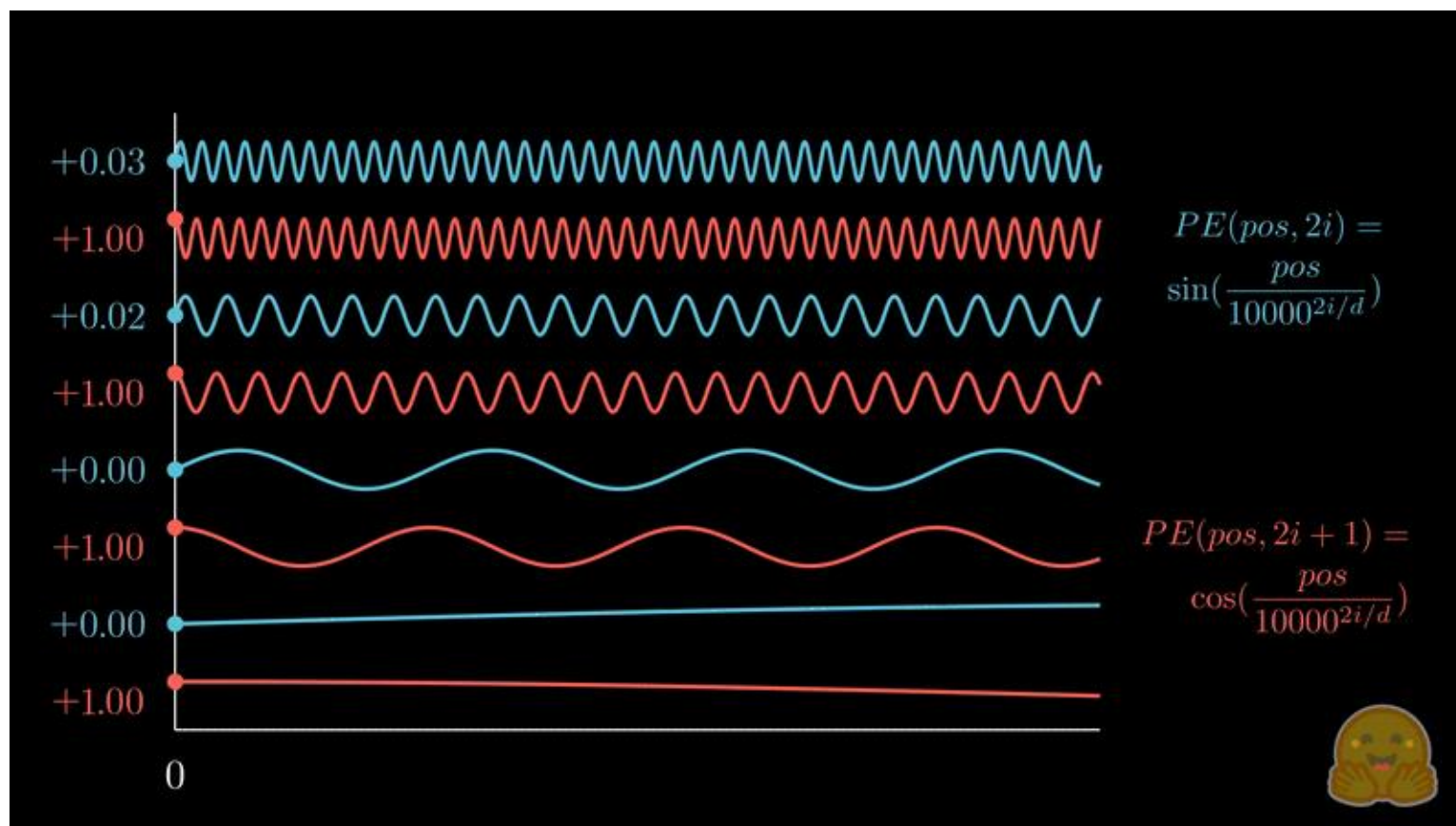


Positional Encoding

- Valores binários:
 - Redes neurais trabalham com valores contínuos
 - Valores binários são discretos

Positional Encoding

- Alternativas? Funções Seno e Cosseno?



Positional Encoding

- Funções Seno e Cosseno :

$$P(k, 2i) = \sin \left(\frac{k}{n^{2i/d}} \right)$$

- Onde:

- K é a posição do token

$$P(k, 2i + 1) = \cos \left(\frac{k}{n^{2i/d}} \right)$$

- d: é a dimensão do embedding

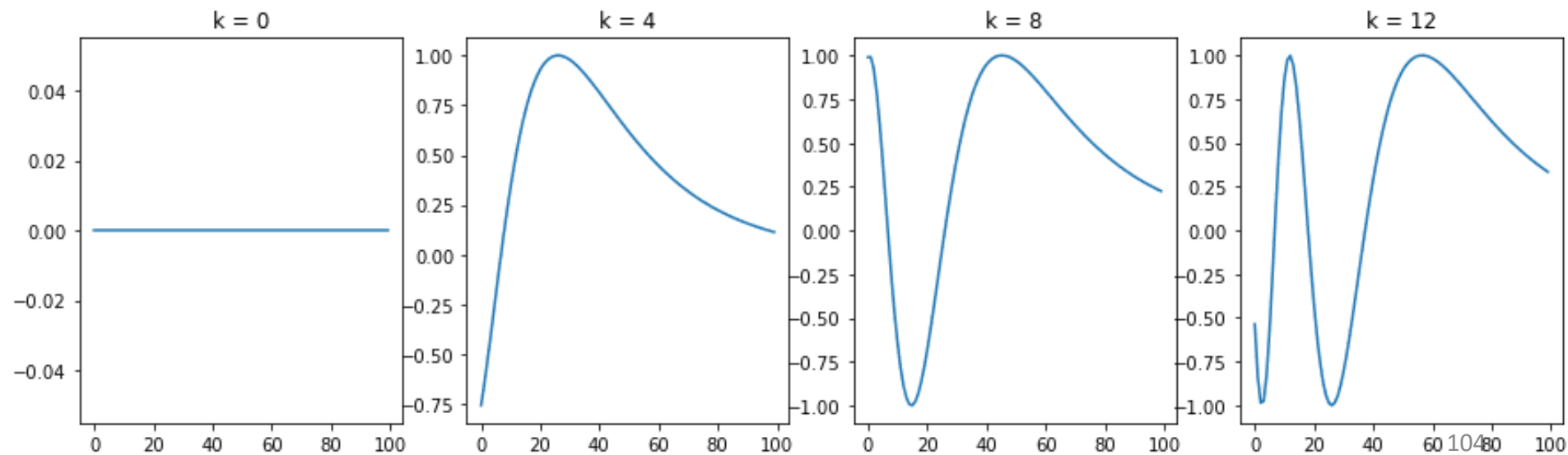
- n: escalar definido pelo usuário, no paper original 10.000

- i: índice da coluna (feature) do embedding

- Usamos essas duas equações para construir embeddings que guardam apenas a **informação posicional**

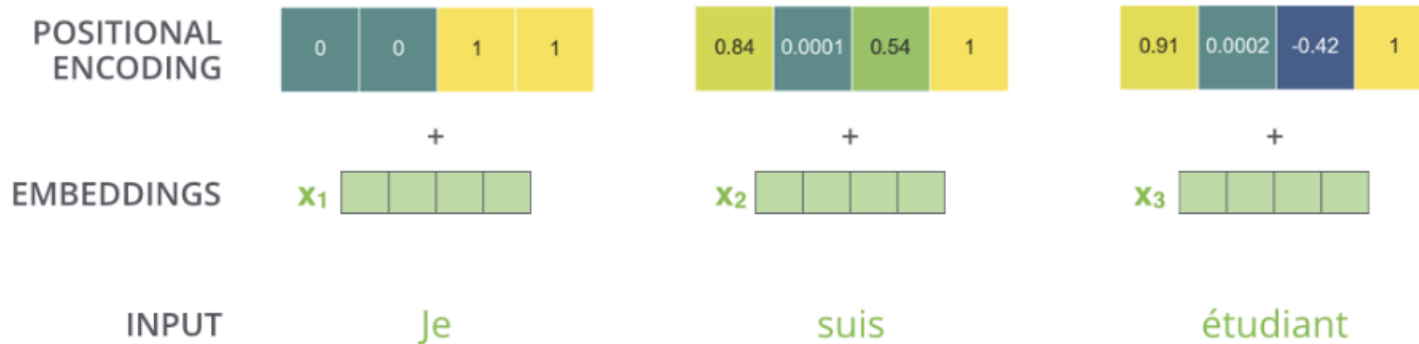
Positional Encoding

- Funções Seno e Cosseno :
 - Dependendo da posição do token, temos uma função sinusoidal diferente
 - $y \rightarrow$ indica o valor
 - $x \rightarrow$ indica a posição no embedding do token k



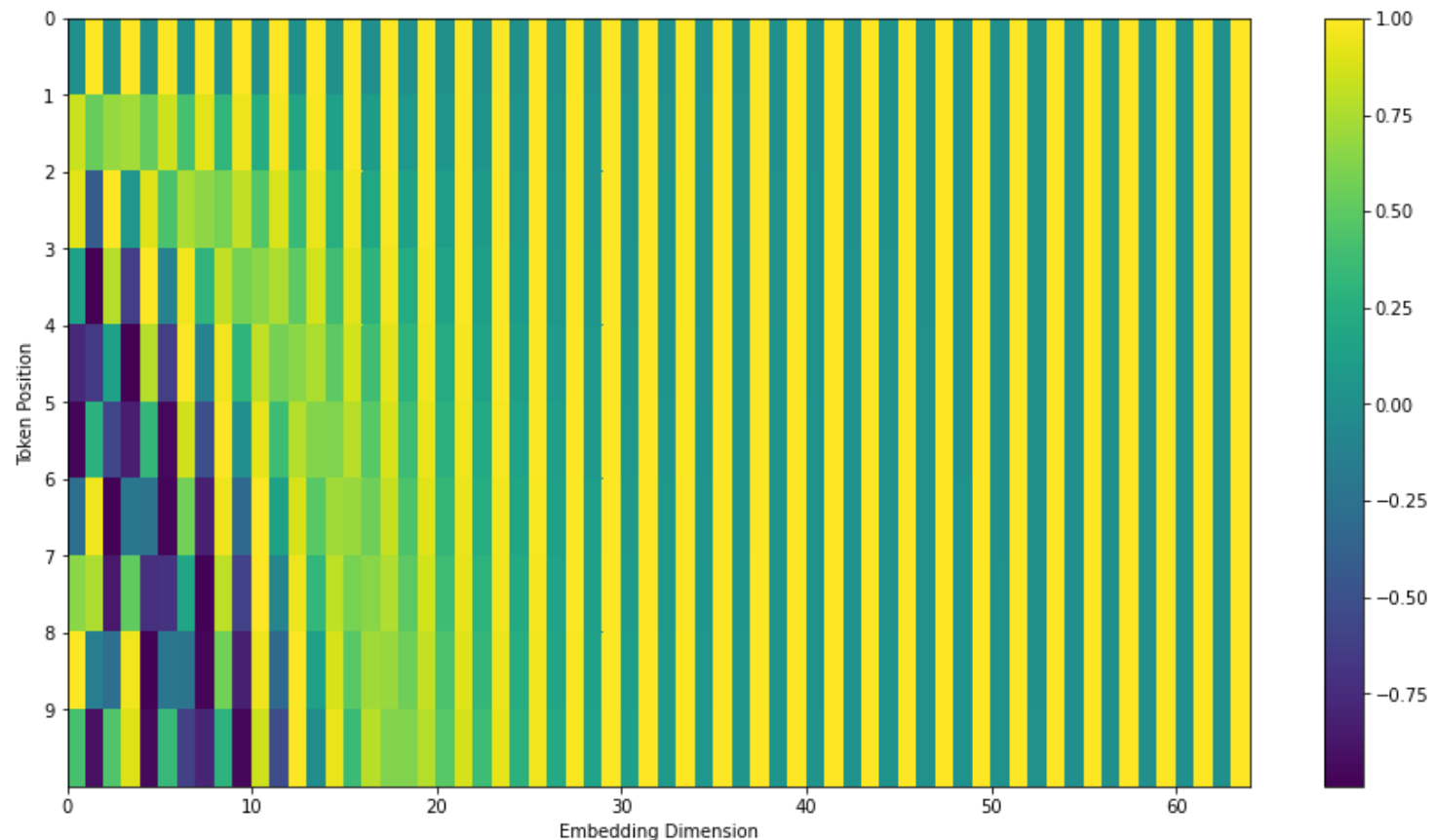
Positional Encoding

- Funções Seno e Cosseno :
 - Visão individual



Positional Encoding

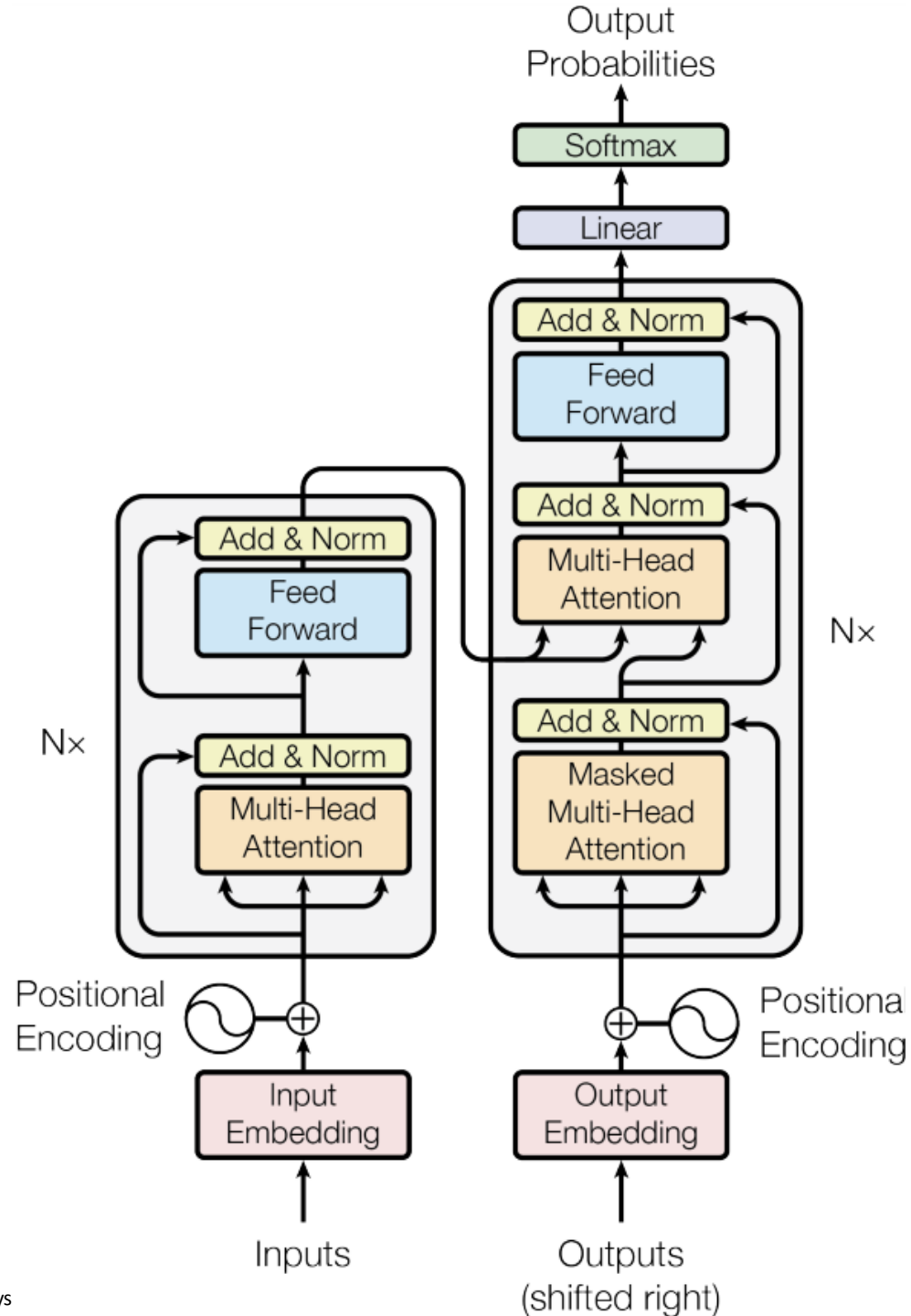
- Funções Seno e Cosseno :
 - Visão geral



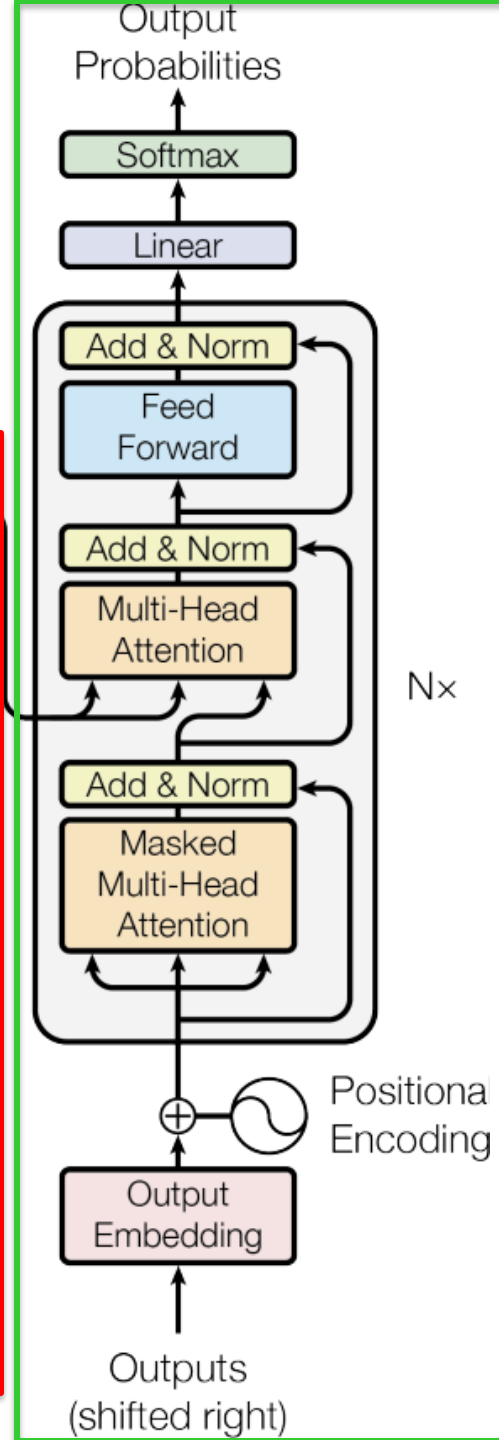
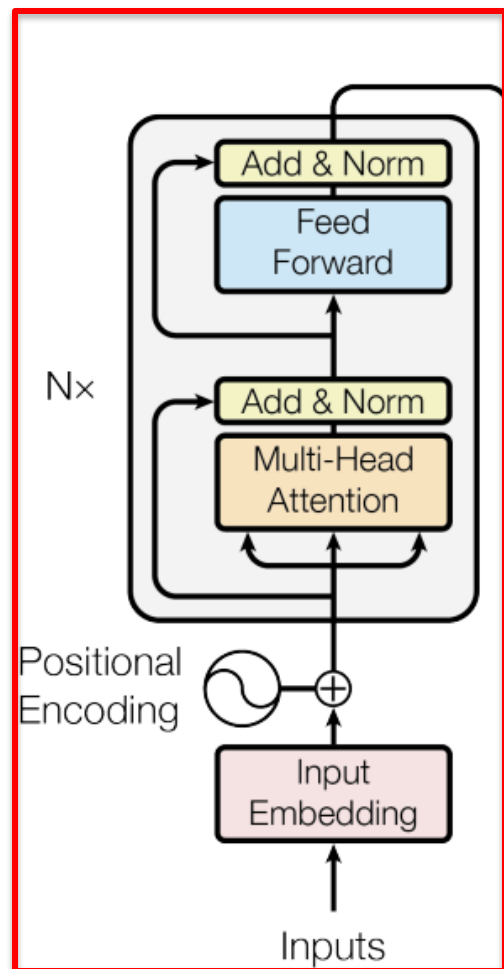
Transformers

- Vamos encararla por partes:

- Self-Attention
- Positional Encoding
- Encoder
- Decoder



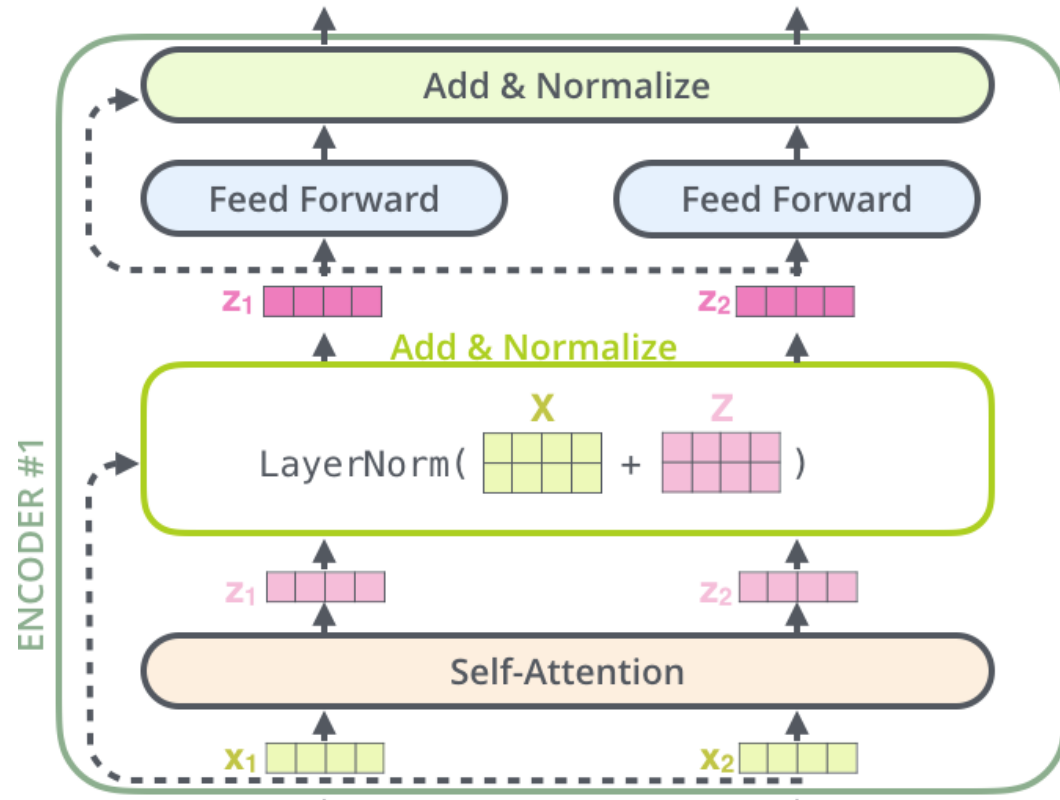
Compreender o
Texto



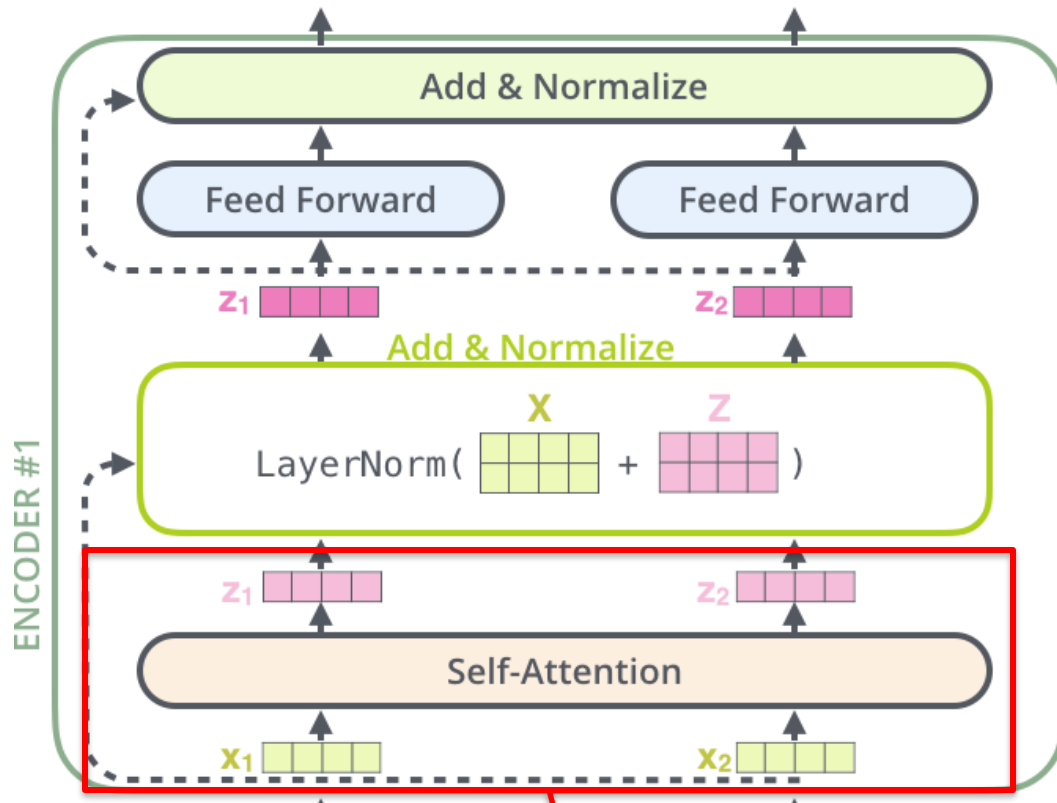
Gerar Texto
Novo

Encoder

- Cada bloco de Encoder é composto por:
 - Self-Attention
 - Conexão Residual
 - LayerNorm
 - Feed-Forward



Encoder

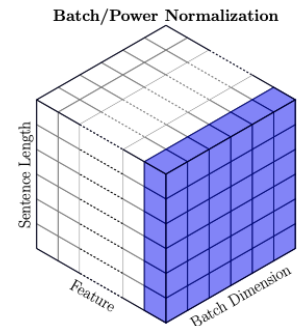
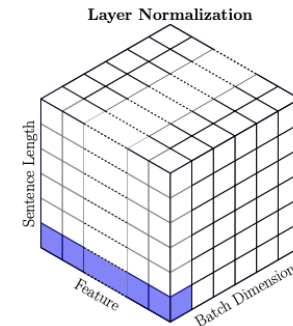
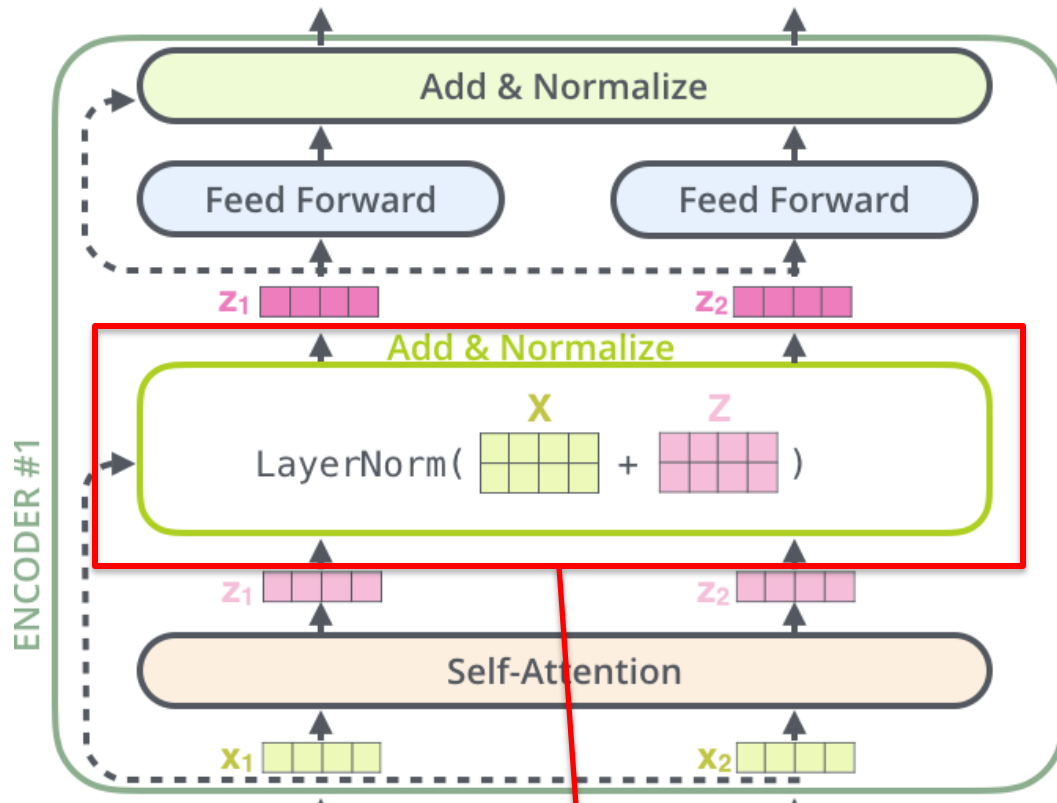


$$z = seq_{size} * emb_{size}$$

$$x = seq_{size} * emb_{size}$$

Este trecho já vimos! Temos uma matriz x como entrada e devolvemos uma matriz z

Encoder



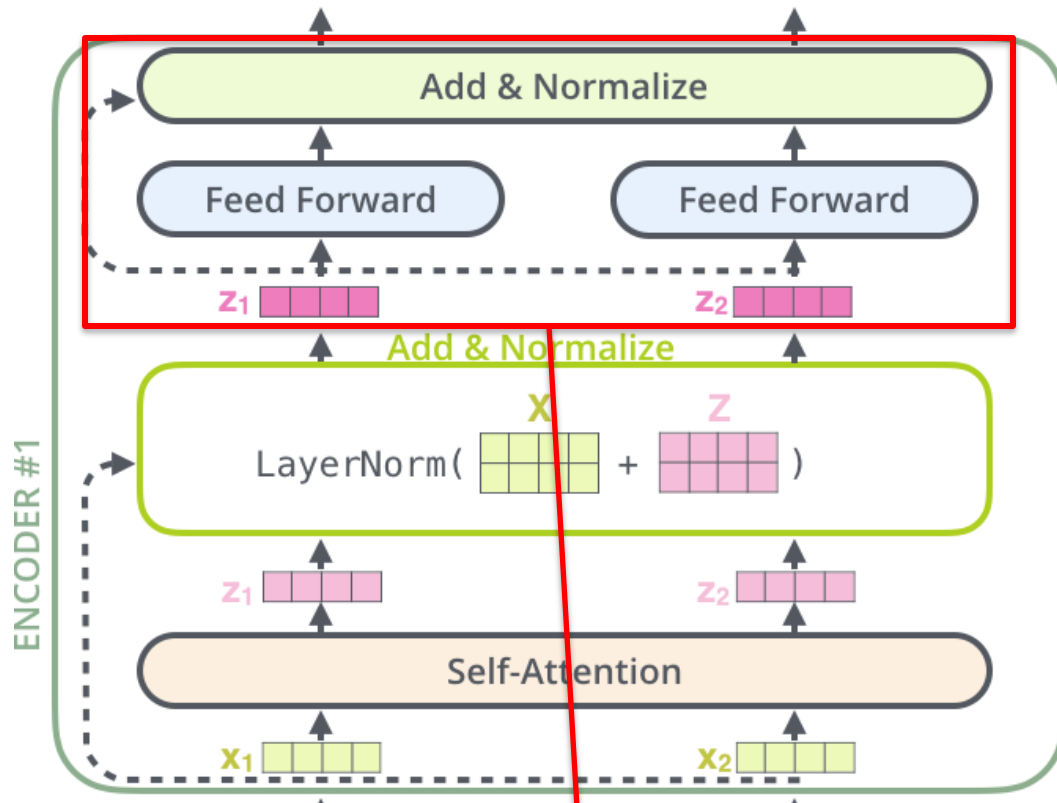
$$z = seq_{size} * emb_{size}$$

$$x = seq_{size} * emb_{size}$$

Neste caso, nós adicionamos a entrada original do encoder com a saída da atenção!

Mesmo shape!

Encoder



$$z = seq_{size} * emb_{size}$$

$$z = seq_{size} * emb_{size}$$

$$x = seq_{size} * emb_{size}$$

Após adicionar a informação
de Self-Attention nos
embeddings originais,
passamos eles para uma MLP

Encoder

É a mesma MLP, mas podemos aplicar em paralelo!

No módulo:

$$\mathbb{R}^{seq_size \times emb_size} \rightarrow \mathbb{R}^{seq_size \times emb_size}$$

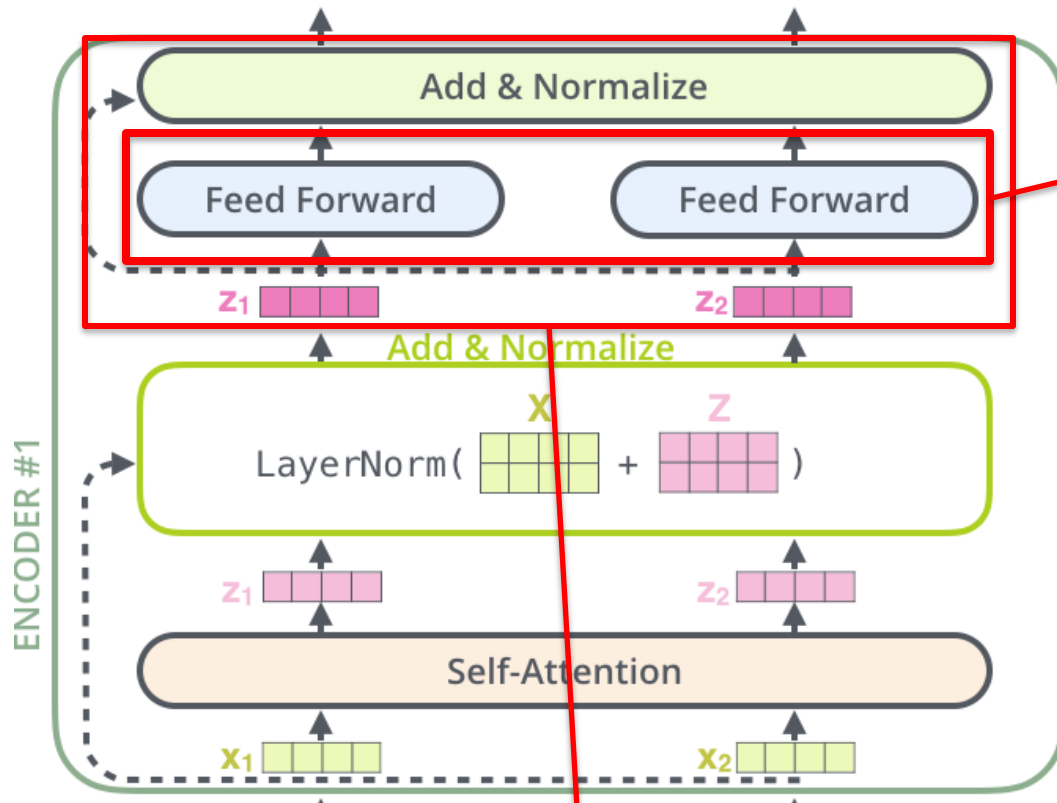
Individualmente:

$$\mathbb{R}^{emb_size} \rightarrow \mathbb{R}^{emb_size}$$

$$z = seq_size * emb_size$$

$$z = seq_size * emb_size$$

$$x = seq_size * emb_size$$

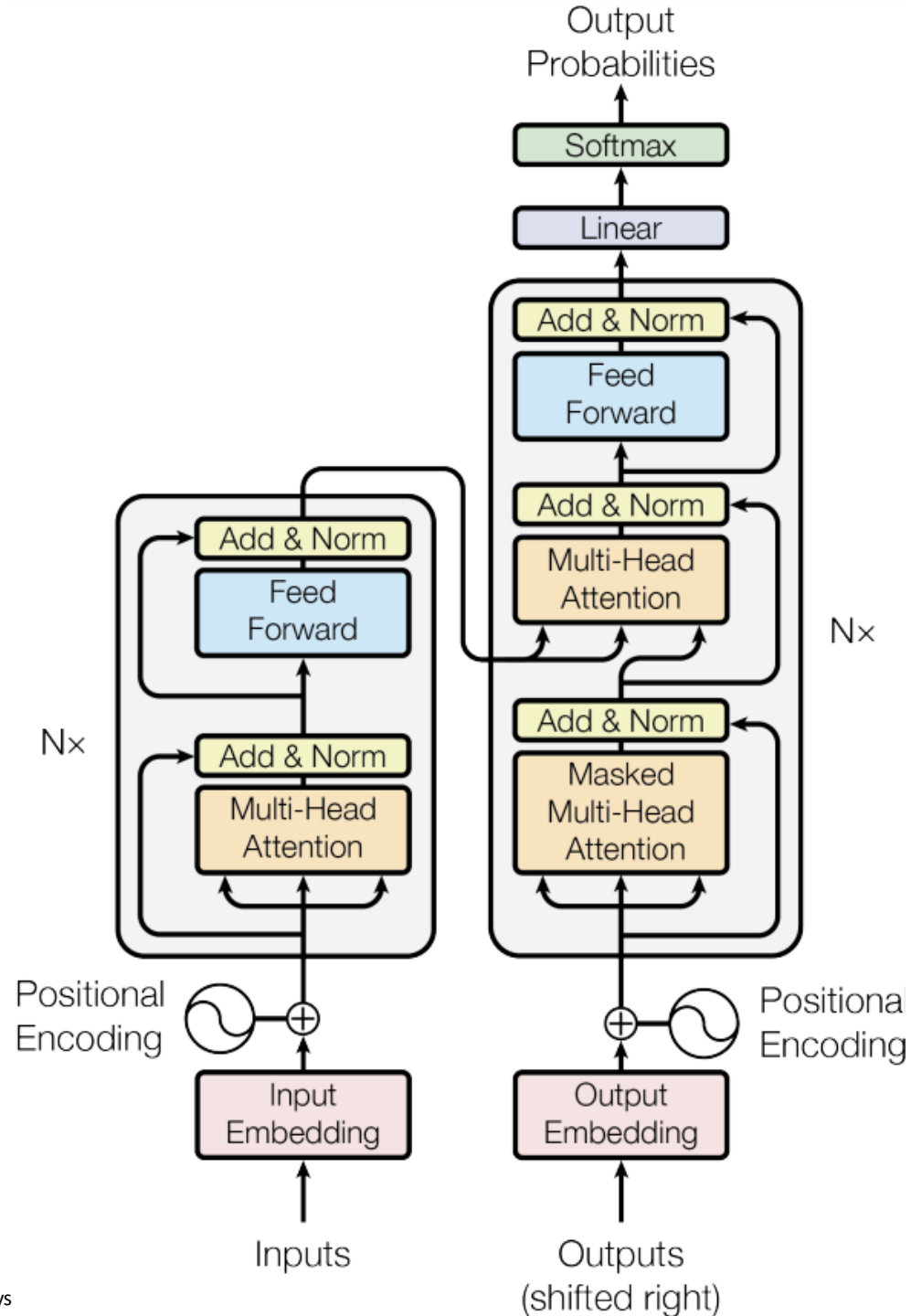


Após adicionar a informação de Self-Attention nos embeddings originais, passamos eles para uma MLP

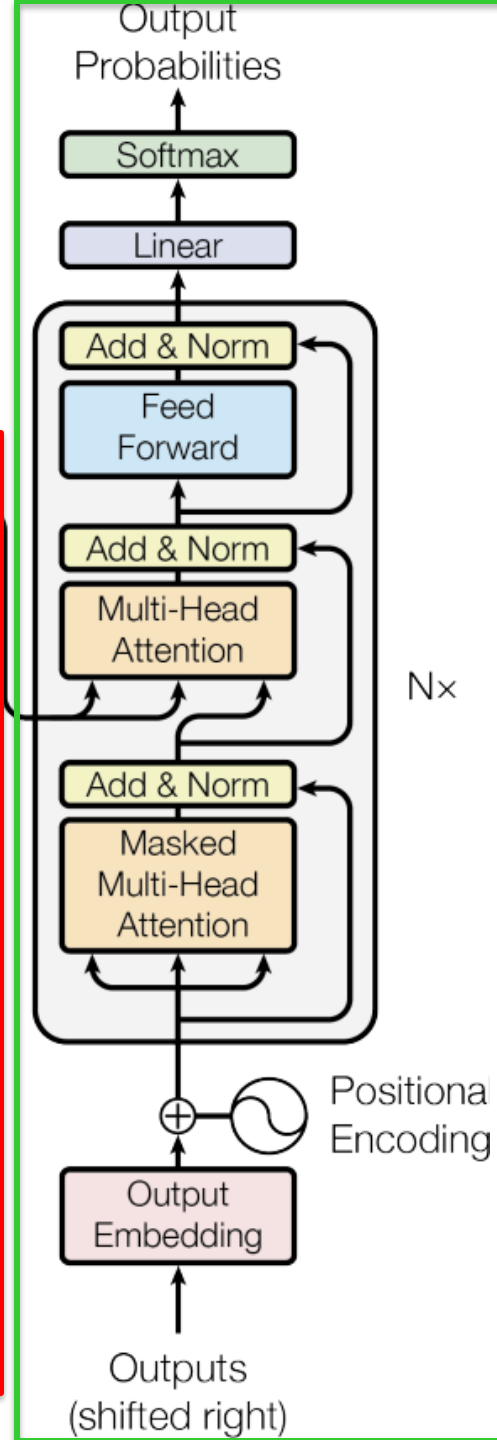
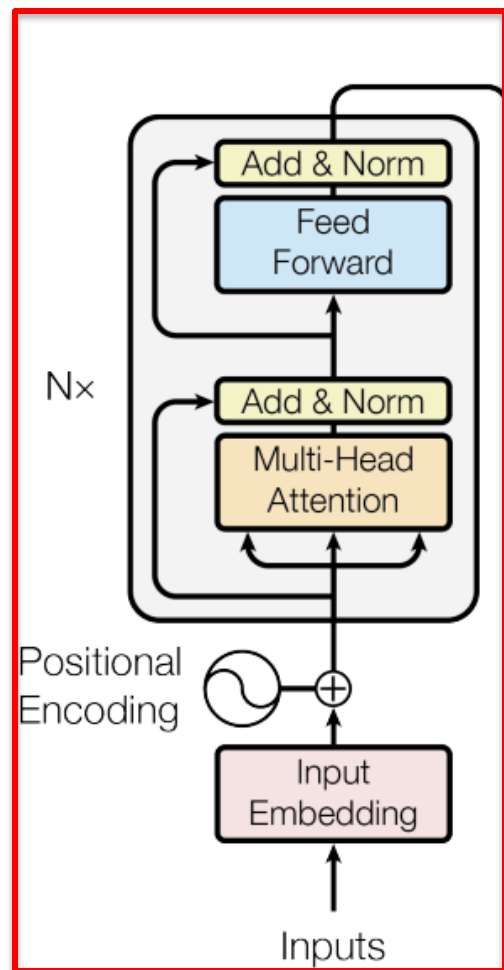
Transformers

- Vamos encará-la por partes:

- Self-Attention
- Positional Encoding
- Encoder
- Decoder



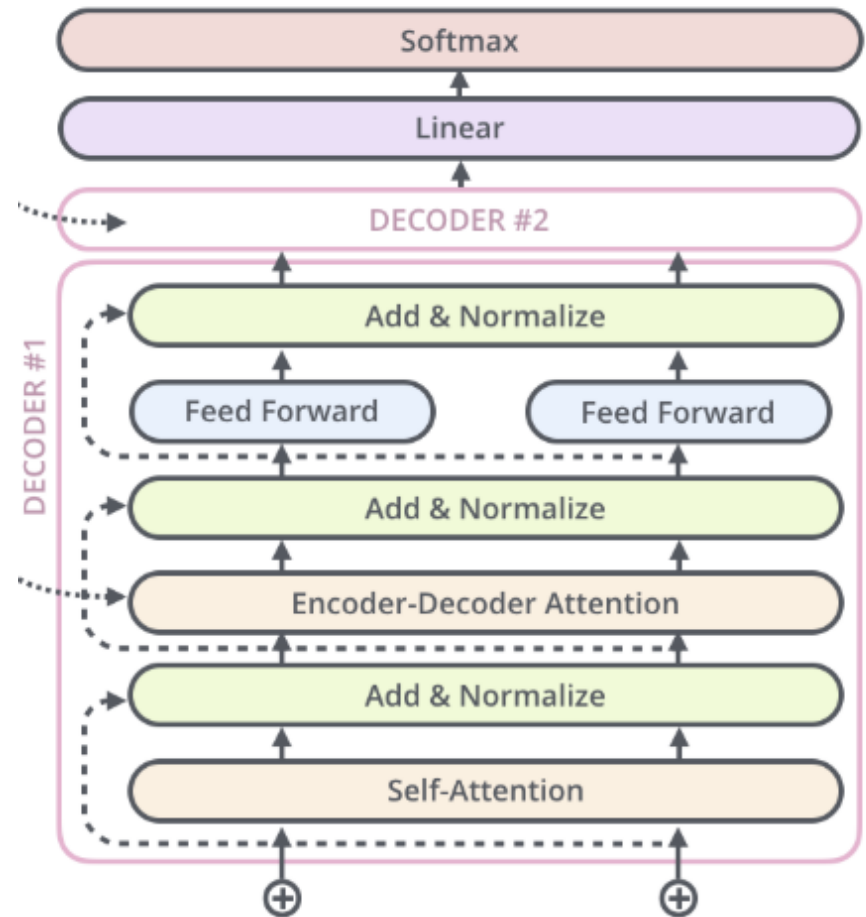
Compreender o
Texto



Gerar Texto
Novo

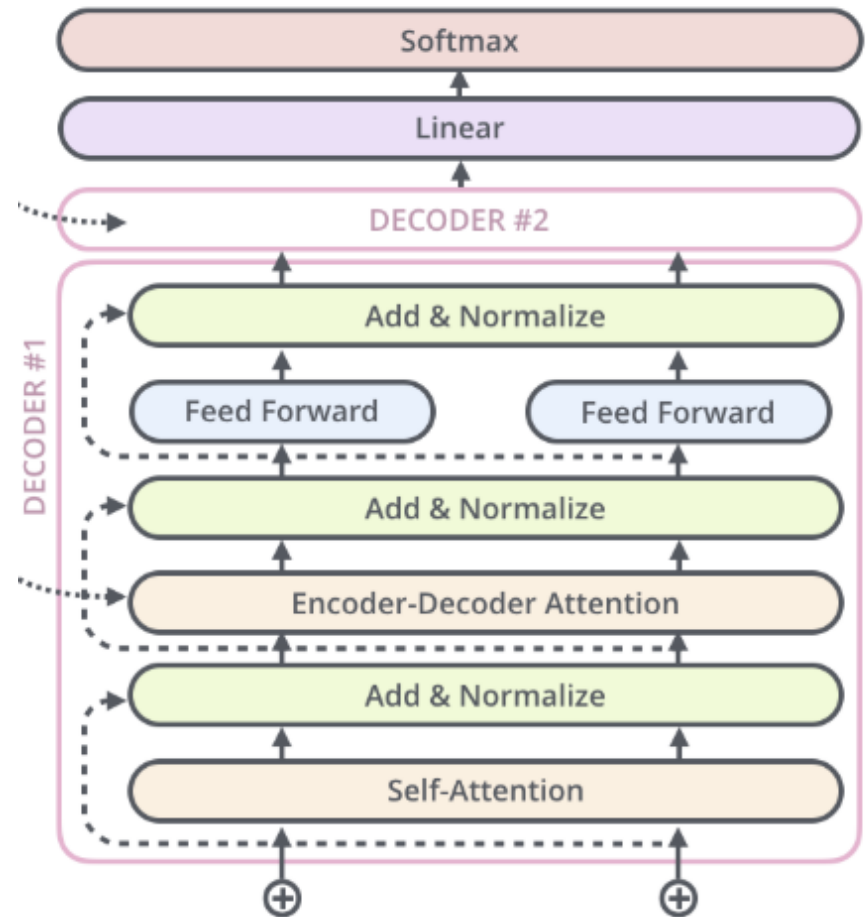
Decoder

- Cada bloco de Decoder é composto por:
 - Masked Self-Attention
 - Conexão Residual
 - LayerNorm
 - Encoder-Decoder Attention
 - Feed-Forward

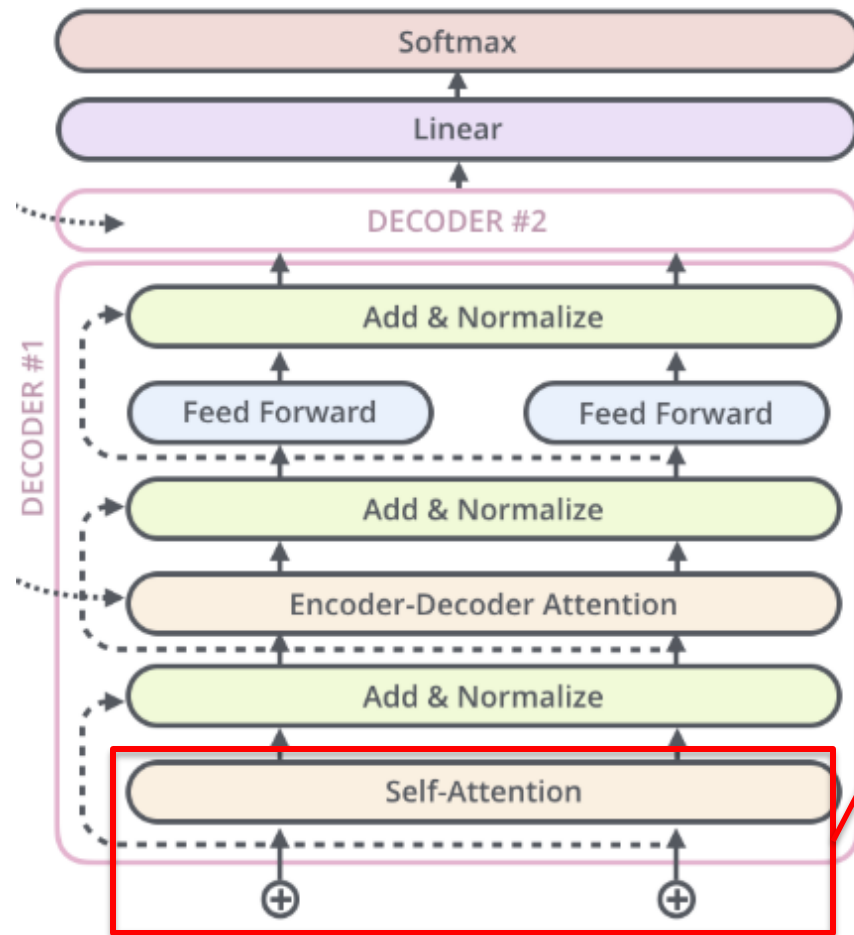


Decoder

- Cada bloco de Decoder é composto por:
 - Masked Self-Attention
 - ~~Conexão Residual~~
 - ~~LayerNorm~~
 - Encoder-Decoder Attention
 - ~~Feed Forward~~

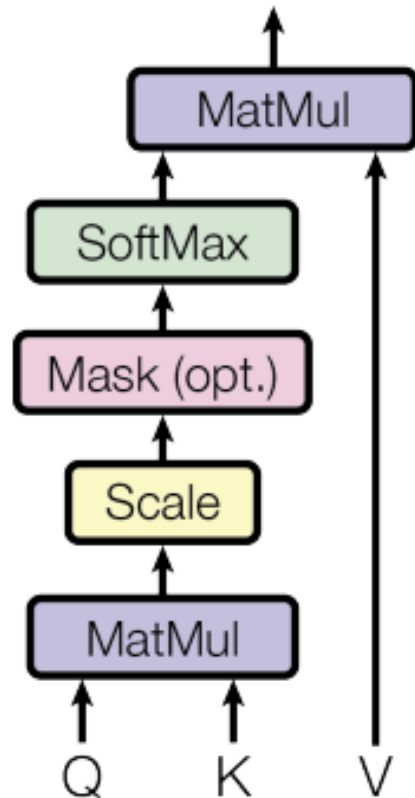


Decoder



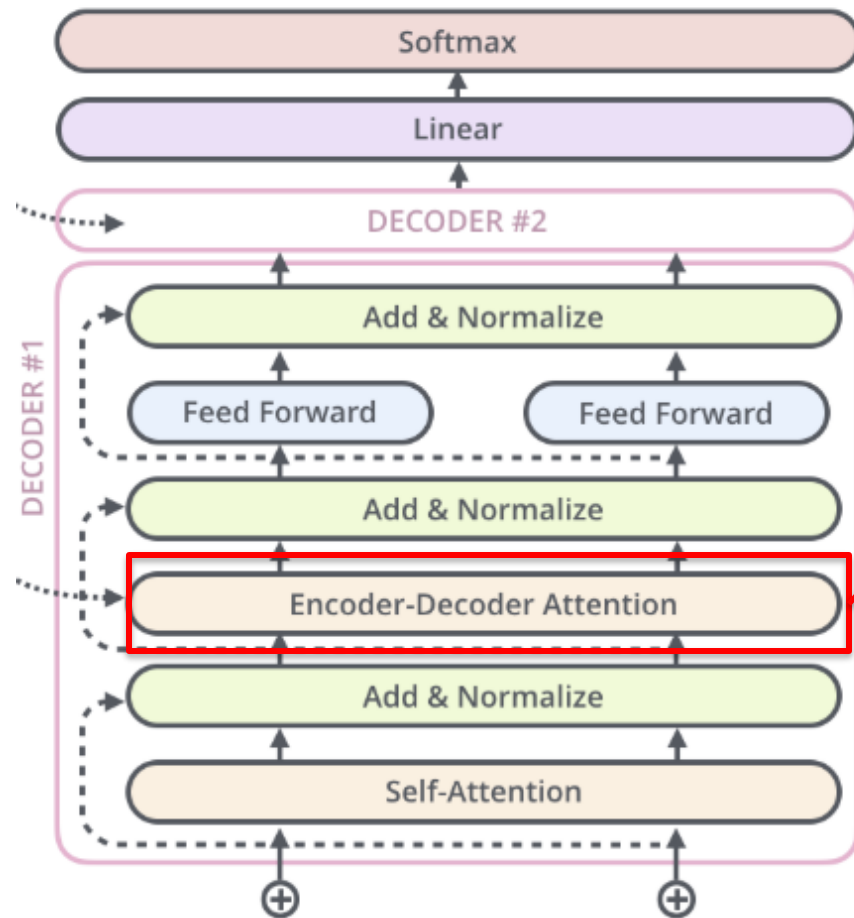
Neste caso entramos com a sequência e aplicamos uma máscara na hora de calcular o *Self-Attention*

Decoder



- Bloco de Self-Attention possui uma **máscara opcional**
- Adicionamos uma máscara nos scores não normalizados
 - 0s e $-\infty$
 - Tokens a **direita do token atual são mascarados por $-\infty$** , dessa forma o peso da atenção é zerado
 - Dessa forma, **impedimos o modelo de ver tokens ainda não gerados**
- A máscara **só existe em tempo de treinamento**, em inferência os tokens à direita de fato não existem

Decoder

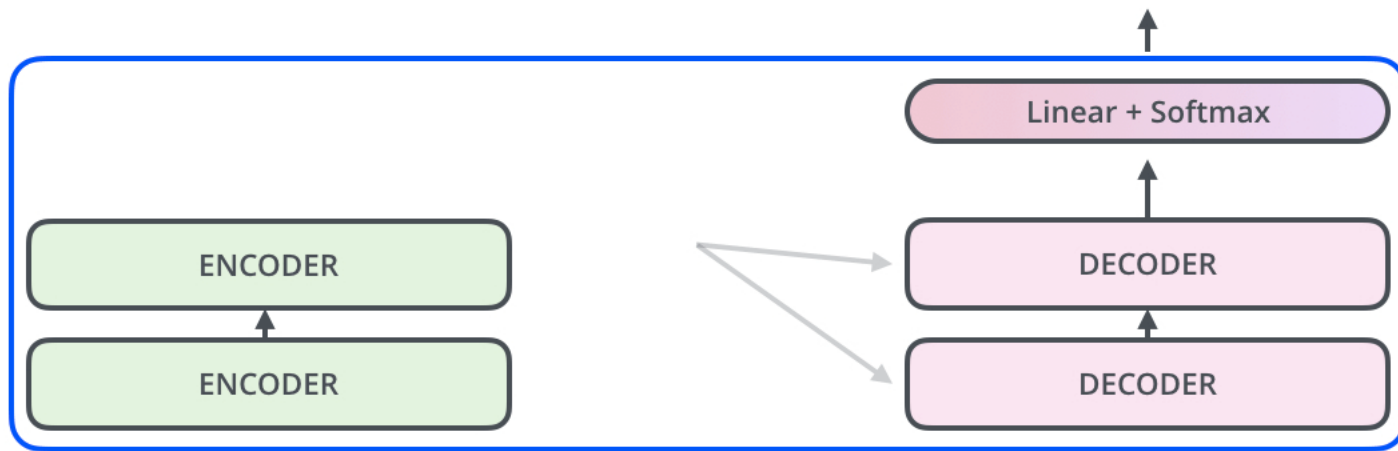


Neste caso, aplicamos *Self-Attention* mas usamos as matrizes de Key e Value do encoder e apenas a Query do decoder

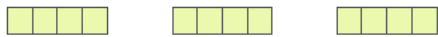
Decoder

Decoding time step: 1 2 3 4 5 6

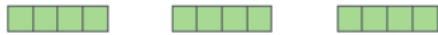
OUTPUT



EMBEDDING
WITH TIME
SIGNAL



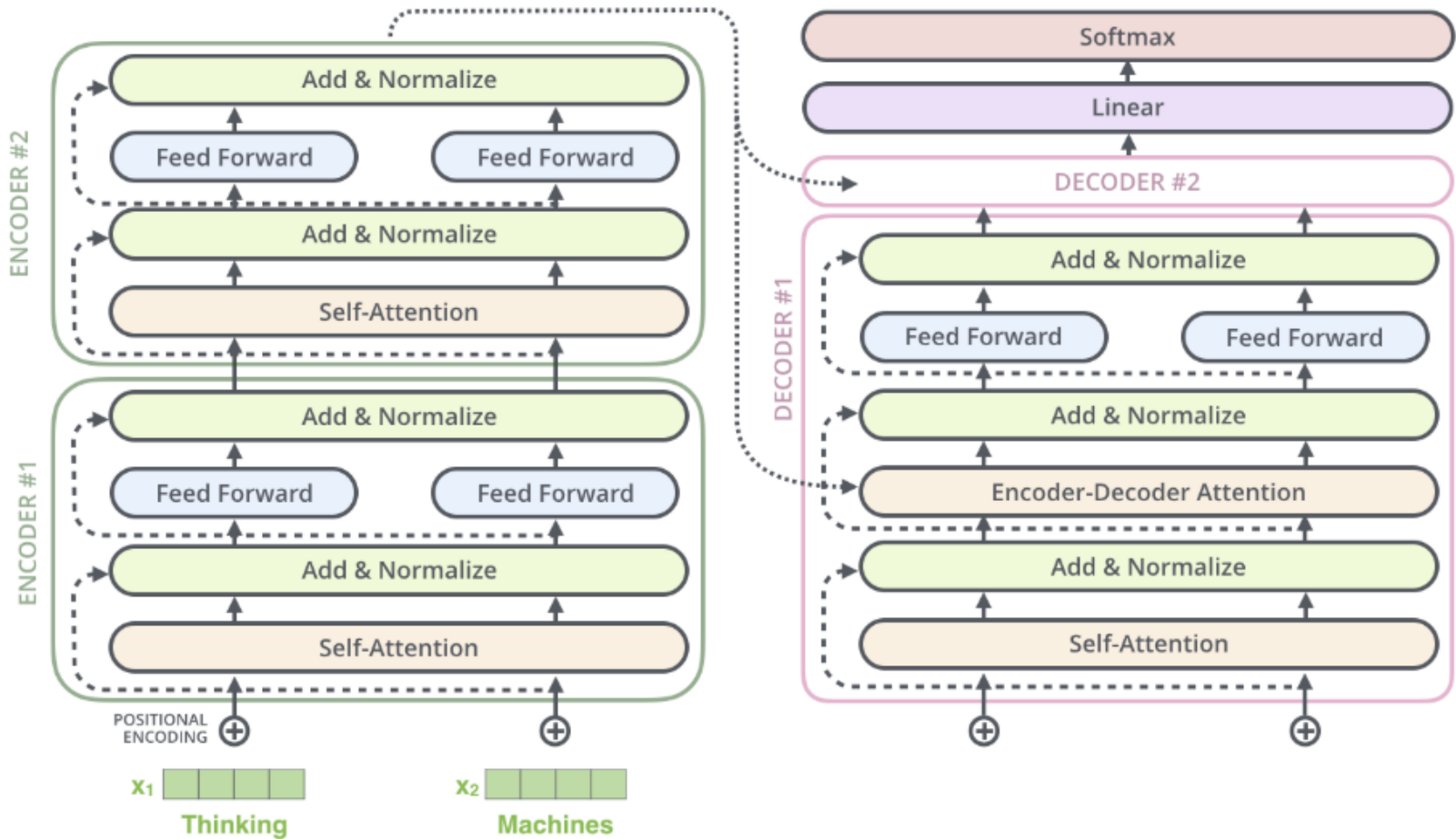
EMBEDDINGS



INPUT

Je suis étudiant

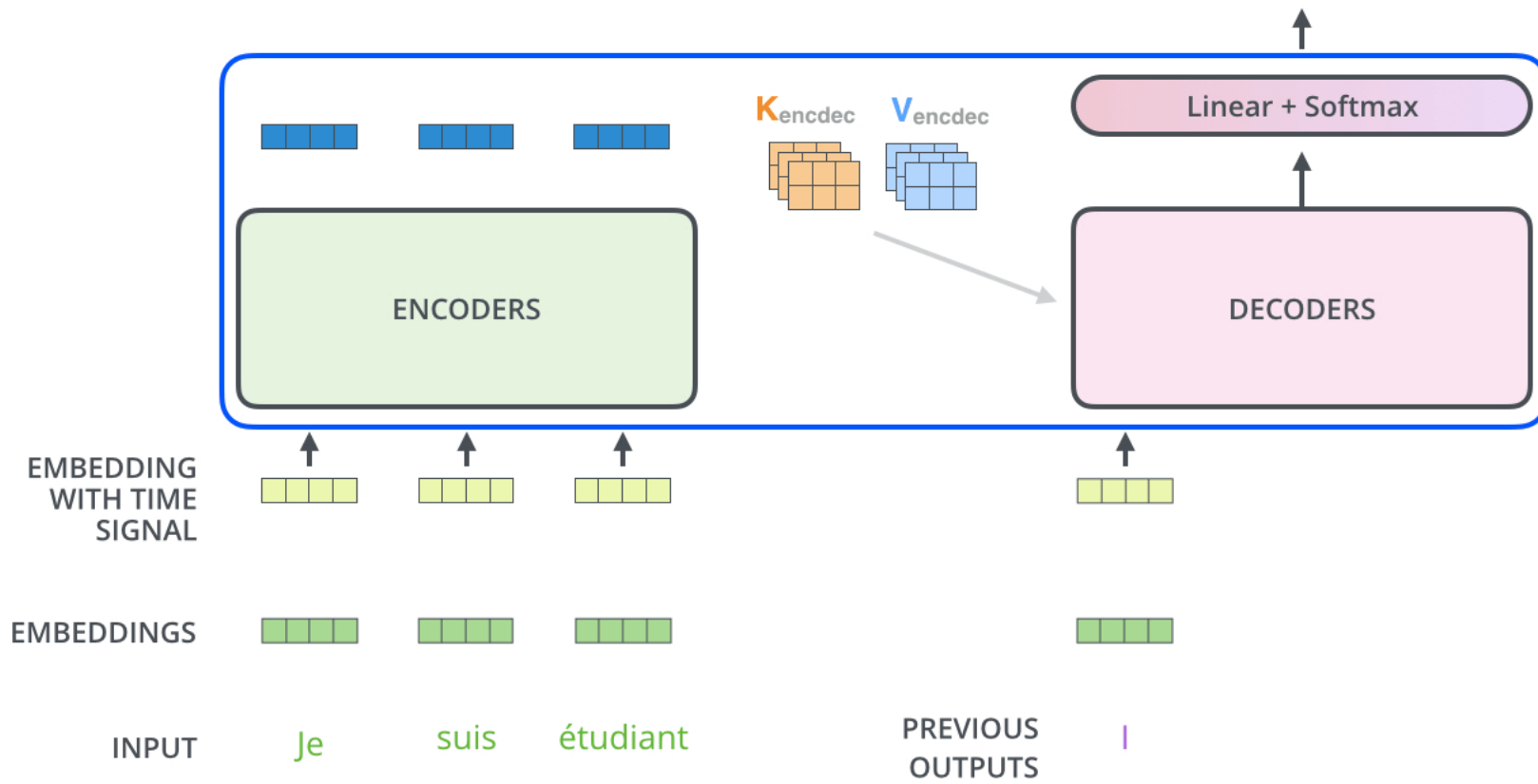
Decoder



Decoder

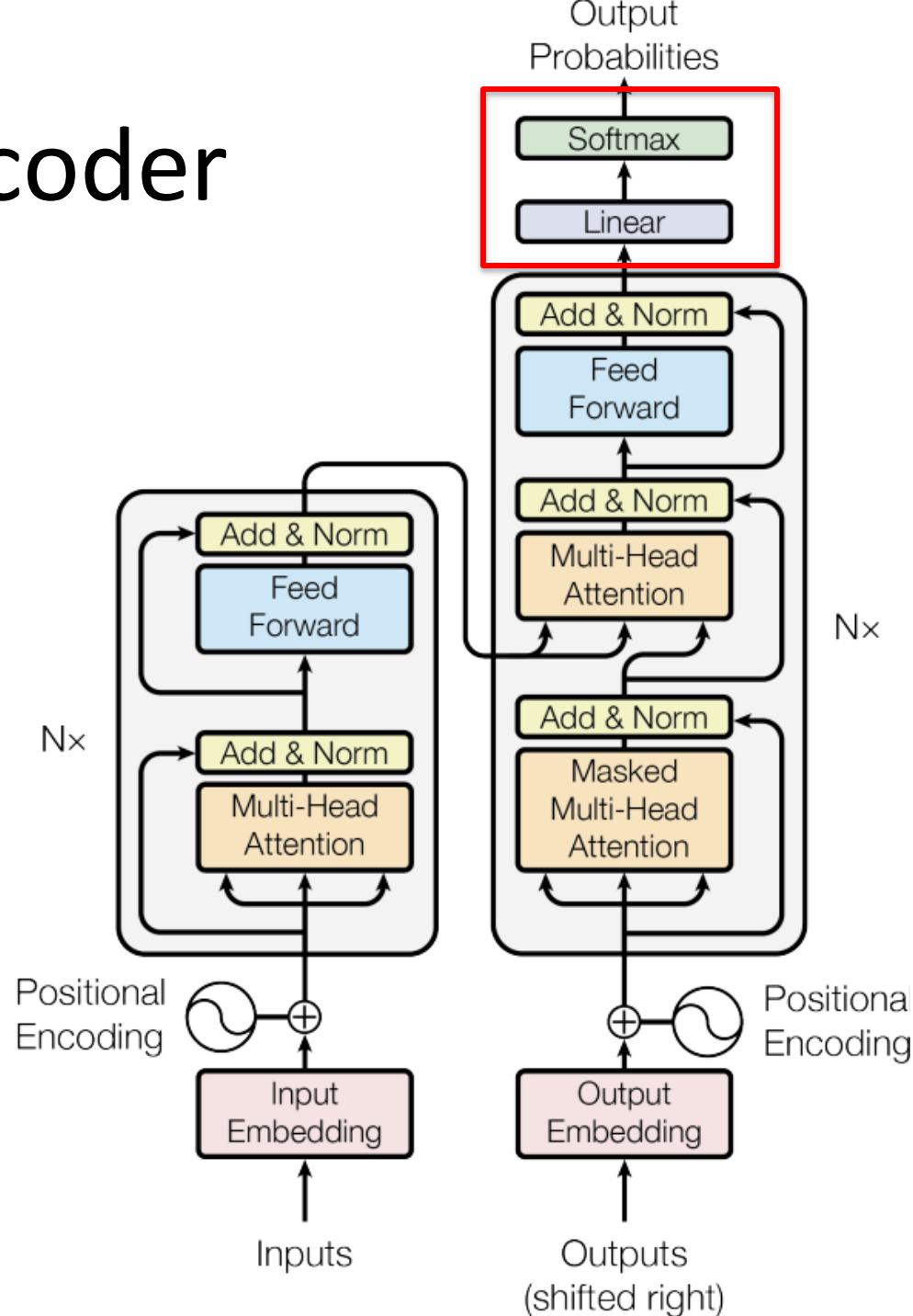
Decoding time step: 1 2 3 4 5 6

OUTPUT |



Decoder

- Ao final, passamos a saída do decoder para uma **camada linear** e uma **softmax**
 - Obtemos qual a palavra mais provável



Transformers

- Vamos encará-la por partes:

- Self-Attention
- Positional Encoding
- Encoder
- Decoder

